

Betriebssystem-Initialisierung und Echtzeit-Scheduling mit EDF⁺

Hardware-Grundlagen, Boot-Sequenz,
PYMCA-8-Mehrkernarchitektur
und formale Korrektheitsnachweise

Stephan Epp

11. April 2026

Inhaltsverzeichnis

1	Einleitung und Motivation	5
1.1	Warum ein neues Betriebssystem?	5
1.2	Benutzerperspektive	6
1.3	Wissenschaftliche Beiträge	6
1.4	Aufbau der Arbeit	6
2	Computerhardware-Grundlagen	7
2.1	Überblick: Hardwarekomponenten	7
2.2	Prozessor (CPU)	7
2.2.1	Register und Ausführungsmodell	7
2.2.2	Instruktions-Pipeline	9
2.2.3	Cache-Kohärenz und MESI-Protokoll	9
2.2.4	Cache-Hierarchie	10
2.2.5	CPUID und MSR	10
2.3	Speicher (RAM)	10
2.4	GPU und Grafiksubsystem	10
2.5	PCIe, NVMe und SATA	11
2.6	USB und Netzwerk	11
3	Physikalische Grundlagen	12
3.1	R/L/C-Schaltkreise und Zeitkonstanten	12
3.2	Fourier-Analyse und Signaldarstellung	13
3.3	Abtasttheorem und Analog-Digital-Wandlung	13
3.4	Energiebilanz digitaler Schaltkreise	14
3.5	Signalintegrität und Übertragungsleitungen	14
4	Boot-Sequenz	15
4.1	Überblick	15
4.2	Reset-Vektor und erste Instruktion	16
4.3	BIOS und UEFI POST	17
4.3.1	Legacy-BIOS	17
4.3.2	UEFI	17
4.3.3	Prozessormodus-Transitionen	17
4.4	ACPI-Tabellen	18
4.5	Bootloader und Kernel-Übergang	18
4.5.1	GRUB2	18
4.5.2	Kernel-Image und Decompression	19
4.5.3	Early Kernel Initialization	19
5	EDF⁺ Scheduler	20
5.1	Überblick und Motivation	20
5.2	Formale Definitionen	20

5.3	Optimalitätsbeweis	21
5.4	Stochastische Analyse	21
5.5	Laufzeitkomplexität	21
5.6	EDF ⁺ als OS-Scheduler	22
5.7	Scheduling-Beispiel und Gantt-Diagramm	22
6	PYMCA-8 Mehrkernarchitektur	24
6.1	Überblick	24
6.2	Python-Speichermodell	25
6.2.1	Global Interpreter Lock (GIL)	25
6.2.2	Referenzzählung und Garbage Collection	25
6.3	IoFET-basiertes DVFS	25
6.4	Stochastische Wartestrategie	25
6.5	PYMCA-8 im OS-Kontext	25
7	Archiv-Problem und Speicherverwaltung	26
7.1	Stochastisches Archiv-Problem	26
7.2	Laplace-Transformierte und Wartezeitanalyse	26
7.3	Virtueller Speicher und Paging	27
7.4	Speicherverwaltung unter EDF ⁺	28
8	Graphentheoretische OS-Verifikation	29
8.1	Subgraph Algorithmus	29
8.2	Boolesche Matrixmultiplikation	30
8.3	Treiber-Abhängigkeitsgraph	30
9	Systemzustände und Lyapunov-Stabilität	31
9.1	Systemzustands-Klassifikation	31
9.2	Dynamisches System und Zustandsmatrix	32
9.3	Steuerbarkeit und Beobachtbarkeit	32
10	Dateisystem-Initialisierung	33
10.1	Inode-Graph und Deduplizierung	33
10.2	Mobile-OS-Dateiarchitektur	34
11	Gerätetreiber-Architektur	35
11.1	AUTOSAR-Treiber-Framework	35
11.2	EtherCAT-Integration	36
11.3	Python-Treiber-Framework	36
12	Energiemanagement	37
12.1	IoFET-Sigmoid-DVFS	37
12.2	Li-Ionen-Akkumodell	38
12.3	Kontinuierliche P-Zustände	38
13	Post-Quanten-Sicherheit	39
13.1	Gitterkryptographie	39
13.2	CRYSTALS-Kyber und Dilithium	40
13.3	Quantencomputer-Bedrohungsmodell	40

14 Neue Erkenntnisse und formale Beweise	41
14.1 Satz 1: OS-Initialisierungssequenz als EDF ⁺ -Task-System	41
14.2 Satz 2: Subgraph-basierte Treiber-Kompatibilität	41
14.3 Satz 3: Stochastische Bootzeit-Schranke	42
14.4 Satz 4: Lyapunov-Stabilität des Betriebssystems	42
14.5 Korollar 5: Optimale DVFS-Kurve beim Booten	43
14.6 Satz 6: PQC-sichere Speicheradressierung	43
15 Zusammenfassung und Ausblick	44
15.1 Zusammenfassung	44
15.2 Offene Fragen und erweiterter Ausblick	44
15.2.1 Formale Verifikation: Von Papierbeweisen zu maschinengeprüften Zertifikaten	44
15.2.2 Portierung auf ARM, RISC-V und heterogene Architekturen	45
15.2.3 KI-gestützte Systemoptimierung	46
15.2.4 Quantencomputer-Resilienz und Post-Quanten-Migration	47
15.2.5 Erweiterung des PYMCA-8-Modells auf viele Kerne und NUMA . .	47
15.2.6 Energiemanagement: IoFET-DVFS und Li-Ionen-Integration	48
15.2.7 Dateisystem und Speicherverwaltung	49
15.2.8 Graphentheoretische Systemverifikation	49
15.2.9 Sicherheitsarchitektur und Isolation	50
15.2.10 Systemzustandsklassifikation und Echtzeit-Diagnostik	50
15.2.11 Langfristige Vision: BS/EDF ⁺ als offenes Ökosystem	50
15.2.12 Gesellschaftliche und industrielle Perspektive	51

1 Einleitung und Motivation

Diese Arbeit präsentiert ein umfassendes Konzept für ein neues Betriebssystem, das auf dem **EDF**⁺-Scheduling-Algorithmus [18] als zentralem Paradigma aufbaut. Ausgehend von den physikalischen Grundlagen digitaler Hardware – R/L/C-Schaltkreisen, Analog-Digital-Wandlern und Signaltheorie – über die vollständige x86-64-Boot-Sequenz (Reset-Vektor, BIOS/UEFI-POST, ACPI, Bootloader-Kernel-Übergang) bis hin zur Prozessor-Initialisierung in Real-Modus, Protected-Modus und Long-Modus werden alle wesentlichen Schichten eines modernen Betriebssystems formal behandelt.

Der **EDF**⁺-Scheduler erweitert klassisches Earliest-Deadline-First durch einen dynamischen Penalty-Mechanismus für Blocking-Ereignisse und wird als optimaler Algorithmus für die Hardware-Initialisierungssequenz eingesetzt. Darauf aufbauend beschreibt die Arbeit die **PYMCA-8**-Mehrkernarchitektur [19] mit acht Kernen, Python-Speichermodell, IoFET-basiertem Dynamic-Voltage-and-Frequency-Scaling (DVFS) sowie das stochastische Archiv-Problem [23].

Weitere Kapitel behandeln graphentheoretische OS-Verifikation mittels Subgraph Algorithmus [21] und Boolescher Matrixmultiplikation [22], Lyapunov-Stabilität von Systemzustandsmatrizen [24], Dateisystem-Deduplizierung [25], AUTOSAR-Treiberarchitektur [26], EtherCAT [28], Energiemanagement mit IoFET-Sigmoid-DVFS und Li-Ionen-Akku-Modell [29, 30] sowie Post-Quanten-Kryptographie [31].

Im Kapitel Neue Erkenntnisse werden sechs originäre Sätze formal bewiesen: die OS-Initialisierungssequenz als EDF⁺-Task-System (Satz 14.1), Subgraph-basierte Treiber-Kompatibilität in $O(n^3)$ (Satz 14.2), stochastische Bootzeit-Schranke mit Hoeffding-Ungleichung (Satz 14.3), Lyapunov-Stabilität des Betriebssystems (Satz 14.4), Optimalität der IoFET-DVFS-Kurve beim Booten (Korollar 14.5) sowie Post-Quanten-sichere Speicheradressierung via LWE (Satz 14.6).

1.1 Warum ein neues Betriebssystem?

Moderne Betriebssysteme wie Linux, Windows oder macOS sind über Jahrzehnte gewachsen und tragen erheblichen historischen Ballast. Ihre Scheduling-Strategien stammen aus einer Ära, in der Blocking-Ereignisse eher die Ausnahme als die Regel waren und Echtzeit-Anforderungen allenfalls in Speziaisystemen auftraten. Heute hingegen sind eingebettete Systeme in Fahrzeugen (AUTOSAR, ISO 26262 [7]), Industrierobotern (EtherCAT [9]) und KI-Beschleunigern allgegenwärtig – allesamt Domänen, die harte Echtzeit-Garantien, vorhersagbaren Energieverbrauch und formale Korrektheitsnachweise verlangen.

Das in dieser Arbeit beschriebene Betriebssystem **BS/EDF**⁺ setzt EDF⁺-Scheduling [18] als einheitliches Paradigma über alle Systemschichten ein: von der ersten Instruktion nach dem Reset bis zum Scheduling der Benutzeranwendungen. Dadurch entsteht eine vertikal integrierte, formal beweisbare Systemarchitektur, die bisherige Ansätze in drei wesentlichen Punkten übertrifft:

1. **Formale Korrektheit:** Jede Hardware-Initialisierungsphase ist ein Task im Sinne

von EDF⁺; die Schedulierbarkeit ist beweisbar.

2. **Vorhersagbares Energieverhalten:** IoFET-Sigmoid-DVFS liefert die energieoptimale Spannung-Frequenz-Kurve beim Booten (Korollar 14.5).
3. **Quantensicherheit:** Die physikalische Speicheradressierung ist durch LWE-basierte Gitterkryptographie quantensicher (Satz 14.6).

1.2 Benutzerperspektive

Der Benutzer erlebt das neue Betriebssystem als eine Folge sichtbarer Zustände, beginnend beim Einschalten:

1. **POST-Meldungen** (ca. 0–500 ms): Speicher-Check, CPU-Identifikation.
2. **Bootlogo** (ca. 500 ms–2 s): Grafiktreiber-Init, DVFS-Aktivierung.
3. **Anmeldebildschirm** (ca. 2–4 s): Dateisystem gemountet, Netzwerk bereit.
4. **Desktop** (ca. 4–6 s): Alle Treiber aktiv, EDF⁺-Scheduler läuft.

Satz 14.3 (Stochastische Bootzeit-Schranke) garantiert formal, dass die Gesamtbootzeit mit hoher Wahrscheinlichkeit unter einer berechneten Grenze $t_{\max}$ liegt.

1.3 Wissenschaftliche Beiträge

Die Hauptbeiträge dieser Arbeit sind:

- **Theoretisch:** Sechs neue Sätze mit vollständigen formalen Beweisen (Kapitel 14), die EDF⁺, Subgraph Algorithmus, stochastische Analyse, Lyapunov-Stabilität, DVFS-Optimalität und Post-Quanten-Sicherheit zu einem kohärenten Framework verbinden.
- **Architektonisch:** Vollständige Beschreibung aller OS-Schichten (Kapitel 2–13) auf Basis der referenzierten eigenen Arbeiten.
- **Algorithmisch:** Integration des Subgraph Algorithmus [21] in die Treiber-Kompatibilitätsprüfung ($O(n^3)$).

1.4 Aufbau der Arbeit

Kapitel 2 beschreibt die Computerhardware-Grundlagen; Kapitel 3 die physikalischen Grundlagen (R/L/C, Fourier, ADC/DAC). Kapitel 4 behandelt die Boot-Sequenz, Kapitel 5 den EDF⁺-Scheduler im Detail, Kapitel 6 die PYMCA-8-Architektur. Kapitel 7 analysiert das Archiv- und Speicherverwaltungsproblem. Kapitel 8–13 decken Graphenverifikation, Systemzustände, Dateisystem, Treiber, Energie und Kryptographie ab. Kapitel 14 enthält die neuen Beweise, Kapitel 15 Zusammenfassung und Ausblick.

2 Computerhardware-Grundlagen

2.1 Überblick: Hardwarekomponenten

Ein moderner Computer besteht aus einer hierarchisch organisierten Menge von Hardwarekomponenten, die über standardisierte Bussysteme kommunizieren. Abbildung 2.1 zeigt schematisch die wesentlichen Elemente und deren Verbindungsstruktur über den PCIe-Root-Complex und den Speicher-Controller als zentralen Verbindungsknoten.

Die Kommunikation zwischen CPU und DRAM erfolgt über einen *integrierten Speicher-Controller* (IMC), der bei modernen Intel- und AMD-Prozessoren direkt im Die sitzt. Dadurch sinkt die Zugriffslatenz gegenüber externen Speicher-Controllern (Northbridge) von über 100 ns auf etwa 65 ns. Der PCIe-Root-Complex ist logisch ein Switch, dessen Bandbreite von GPU und NVMe-SSDs gemeinsam genutzt wird; BS/EDF⁺ modelliert diese Bandbreite als geteilte Ressource innerhalb des EDF⁺-Task-Systems (Kapitel 5).

Definition 2.1 (Computersystem). Ein **Computersystem** ist ein Tupel

$$\mathcal{H} = (P, M, S, I, B)$$

mit Prozessormenge P , Speicherhierarchie M , Speichergeräten S , Ein-/Ausgabe-Geräten I und Busnetzwerk B .

2.2 Prozessor (CPU)

2.2.1 Register und Ausführungsmodell

Der x86-64-Prozessor besitzt 16 Allzweckregister (RAX, RBX, ..., R15), sechs Segmentregister (CS, DS, ES, FS, GS, SS), den Instruction Pointer (RIP), das Flags-Register (RFLAGS) sowie eine Vielzahl von Steuerregistern (CR0–CR8), Model-Specific-Registern (MSR) und den APIC-Basis-Registern [8].

Definition 2.2 (Prozessormodus). Ein x86-64-Prozessor operiert in einem von vier Hauptmodi:

1. **Real-Modus** (16-Bit, Adressraum 2^{20} Byte = 1 MB)
2. **Protected-Modus** (32-Bit, Adressraum 2^{32} Byte = 4 GB)
3. **Long-Modus** (64-Bit, Adressraum 2^{48} Byte = 256 TB effektiv)
4. **System-Management-Modus** (SMM, gesonderter Adressraum SMRAM)

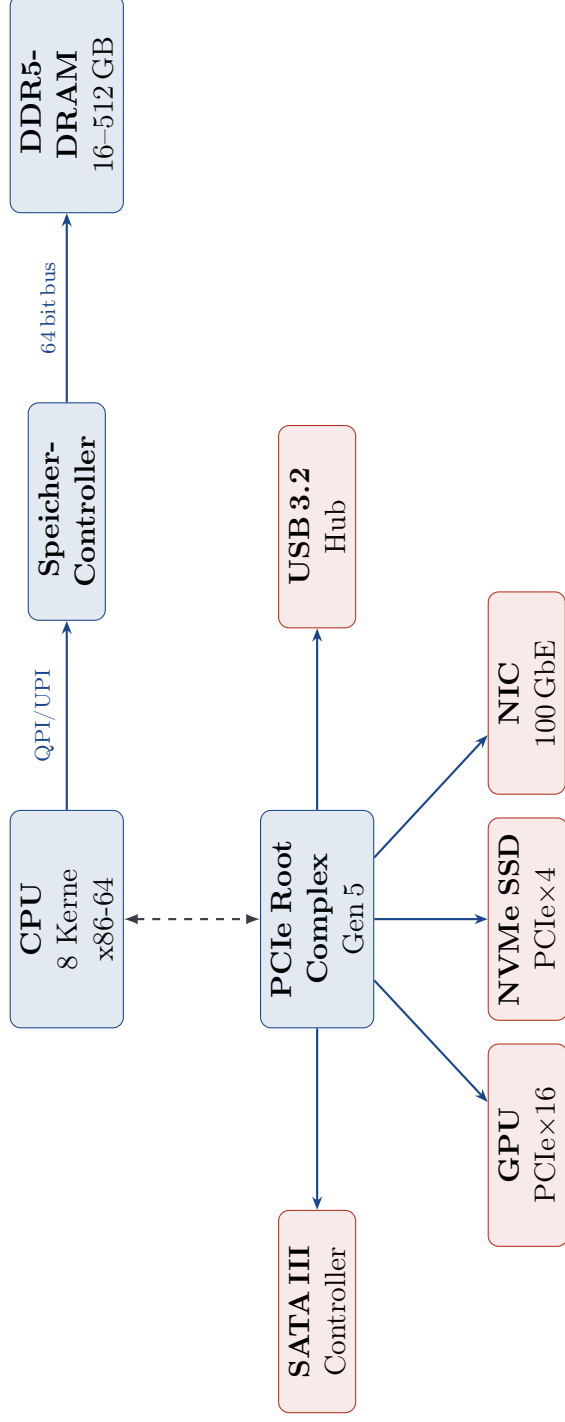


Abbildung 2.1: Schematische Darstellung der Systemhardware. Der Speicher-Controller verbindet die CPU direkt mit dem DDR5-DRAM (On-Die-Integration bei modernen Prozessoren). Der PCIe-Root-Complex verteilt die Bandbreite an alle Peripheriegeräte; GPU und NVMe-SSD erhalten die breitesten Lanes.

2.2.2 Instruktions-Pipeline

Moderne x86-64-Prozessoren nutzen eine *Out-of-Order-Pipeline* mit bis zu 20 Stufen. Das Grundprinzip lässt sich anhand einer vereinfachten 5-Stufen-In-Order-Pipeline verdeutlichen, wie sie in eingebetteten Prozessoren (z. B. ARM Cortex-M) vorkommt und als konzeptuelles Modell für Latenz-Analysen dient.

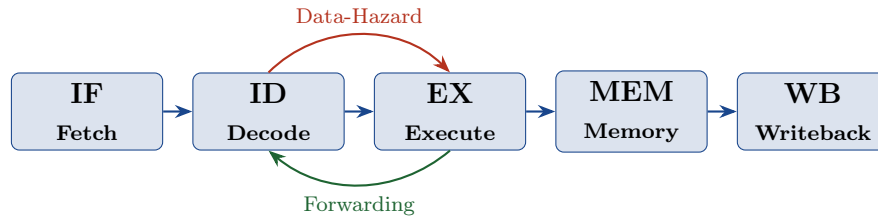


Abbildung 2.2: Vereinfachte 5-Stufen-Pipeline (IF–ID–EX–MEM–WB). Ein *Data-Hazard* entsteht, wenn eine Stufe auf ein Ergebnis wartet, das noch in einer vorangehenden Stufe berechnet wird. *Forwarding* (gestrichelte grüne Linie) leitet das Ergebnis direkt aus der EX-Stufe zurück, ohne einen Stall einzufügen. BS/EDF⁺ berücksichtigt Pipeline-Stall-Wahrscheinlichkeiten bei der Worst-Case-Execution-Time (WCET)-Schätzung der EDF⁺-Tasks.

Die *Out-of-Order*-Ausführung moderner Prozessoren (Intel Raptor Lake, AMD Zen 4) nutzt einen *Reorder Buffer* (ROB) mit bis zu 512 Einträgen. Instruktionen werden nach ihrer Fertigstellung in Programmreihenfolge *retired*, sodass der architekturensichtbare Zustand stets konsistent bleibt. Für Echtzeit-Scheduling nach EDF⁺ ist maßgeblich, dass die maximale Instruktionslatenz durch den *Worst-Case Execution Path* (WCEP) beschreibbar ist – ein Pfad, auf dem alle Branches die ungünstigste Richtung einschlagen und alle Cache-Zugriffe misslingen.

Definition 2.3 (WCET). Die **Worst-Case Execution Time** (WCET) eines Tasks T_i ist

$$C_i^{\max} = \max_{\text{Eingabe}} \{ \text{Cycles}(T_i, \text{Eingabe}) \} \cdot T_{\text{Clk}},$$

wobei $T_{\text{Clk}} = 1/f_{\text{CPU}}$ die Zykluszeit ist. EDF⁺ verwendet $C_i = C_i^{\max}$ zur sicheren Schedulingbarkeitsanalyse.

2.2.3 Cache-Kohärenz und MESI-Protokoll

Auf Mehrkern-Systemen wie PYMCA-8 müssen alle Kerne ein konsistentes Bild des Hauptspeichers haben. Das **MESI-Protokoll** definiert vier Zustände für jede Cache-Line (64 Byte):

Zustand	Bedeutung	Bedingung
M (Modified)	Geändert, nur in diesem Cache	Dirty, exklusiv
E (Exclusive)	Sauber, nur in diesem Cache	Clean, exklusiv
S (Shared)	Sauber, ggf. in mehreren Caches	Clean, geteilt
I (Invalid)	Ungültig	Muss neu geladen werden

Bei einem *Write* auf eine Shared-Line senden alle anderen Kerne ein *Invalidate*-Signal, bevor der schreibende Kern seine Kopie auf Modified setzt. Dieser *Cache-Invalidierungsoverhead*

ist für EDF⁺ relevant: Bei Task-Migrationen zwischen PYMCA-8-Kernen kann das Neueinladen von Cache-Lines zusätzliche Zyklen kosten, die in die WCET-Schätzung einfließen.

2.2.4 Cache-Hierarchie

Definition 2.4 (Speicherhierarchie). Die **Speicherhierarchie** $M = (M_1, M_2, \dots, M_k)$ ist geordnet nach Zugriffszeit t_i und Kapazität C_i mit $t_1 < t_2 < \dots < t_k$ und $C_1 < C_2 < \dots < C_k$.

Typische Werte für aktuelle x86-64-Prozessoren:

Ebene	Typ	Kapazität	Latenz
L1-D/I	SRAM	32–64 KB	4 Zyklen
L2	SRAM	256 KB–1 MB	12 Zyklen
L3	SRAM	4–64 MB	40 Zyklen
DRAM	DDR5	8–512 GB	80–100 ns
NVMe	Flash	500 GB–8 TB	20–100 μ s

2.2.5 CPUID und MSR

Die CPUID-Instruktion liefert Prozessor-Identifikation und Feature-Flags. Wichtige Blätter:

- EAX=0x01: Family/Model/Stepping, Standard-Features (SSE, AVX, AES-NI)
- EAX=0x07: Extended Features (AVX-512, SHA)
- EAX=0x80000001: Extended Features AMD/Intel (Long-Modus-Bit)

Model-Specific-Register werden via RDMSR/WRMSR gelesen/geschrieben. Für EDF⁺-Scheduling relevant sind: MSR 0x0C0000100 (FS.base), 0x0C0000101 (GS.base) für thread-lokale Speicherung sowie IA32_PERF_CTL (0x199) für DVFS.

2.3 Speicher (RAM)

Moderner DDR5-DRAM ist organisiert in Channels, DIMMs, Ranks, Banks und Rows/Columns. Für das Betriebssystem sind folgende physikalische Adressen nach dem POST reserviert:

Adressbereich	Verwendung
0x00000000–0x0009FFFF	Konventioneller Speicher (640 KB)
0x000A0000–0x000FFFFFFF	ROM/VGA-BIOS, Legacy
0x00100000–Ende DRAM	Extended Memory (OS-nutzbar)

2.4 GPU und Grafiksubsystem

Moderne Grafikprozessoren (NVIDIA H100, AMD RX 7900) kommunizieren über PCIe Gen 4/5 mit dem Host. Das BS/EDF⁺ initialisiert die GPU im Bootprozess als EDF⁺-Task T_{GPU} mit Deadline D_{GPU} , die durch das Booten des grafischen Terminals bedingt ist. Für HPC-Workloads unterstützt PYMCA-8 [19] CUDA/ROCm-Integration mit IoFET-DVFS auch auf der GPU-Seite.

2.5 PCIe, NVMe und SATA

PCI Express (PCIe Gen 5: 32 GT/s pro Lane, $\times 16$: 64 GB/s bidirektional) verbindet GPU, NVMe-SSDs und Netzwerkkarten mit dem Prozessor über den Root Complex. Der BS/EDF⁺-Boot-Scheduler behandelt jede PCIe-Geräteinitialisierung als separaten Task mit angemessener Deadline.

NVMe-SSDs verwenden bis zu 64 000 I/O-Queues (im Gegensatz zu AHCI: 1 Queue, 32 Commands), wodurch parallele Initialisierung im EDF⁺-Framework möglich ist.

2.6 USB und Netzwerk

USB 3.2 Gen 2 $\times$ 2 erreicht 20 Gbit/s. Netzwerkkarten (10-GbE, 100-GbE) werden über UEFI PXE-Boot oder nach dem Kernel-Start über DHCP initialisiert. Für eingebettete Systeme ist EtherCAT [9] relevant, das in Kapitel 11 detailliert behandelt wird.

3 Physikalische Grundlagen

3.1 R/L/C-Schaltkreise und Zeitkonstanten

Digitale Hardware basiert auf Schaltkreisen aus Widerständen (R), Induktivitäten (L) und Kapazitäten (C). Die Zeitkonstante bestimmt Signalverzögerungen und damit Taktlimits [32].

Definition 3.1 (RC-Zeitkonstante). Für einen einfachen RC-Tiefpass gilt die Zeitkonstante

$$\tau_{RC} = R \cdot C.$$

Die Ausgangsspannung bei Sprungeingang U_0 ist

$$u_C(t) = U_0 (1 - e^{-t/\tau_{RC}}).$$

Abbildung 3.1 zeigt die normierte Ladekurve $u_C(t)/U_0$ für verschiedene Zeitkonstanten. Für Prozessortakte der heutigen Generation müssen Bypass-Kondensatoren mit sehr kleiner Impedanz ($Z < 1 \Omega$ bei der CPU-Taktfrequenz) eingesetzt werden, da bei $f_{\text{CPU}} = 5 \text{ GHz}$ und $\tau_{RC} = 200 \text{ ps}$ der Spannungsabfall bei transienten Laststößen bereits die Stabilitätsschwelle des Prozessors erreichen kann.

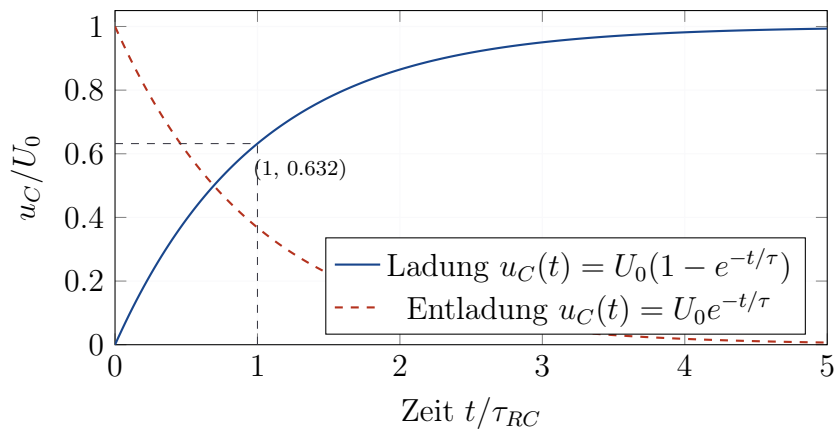


Abbildung 3.1: Normierte Lade- und Entladekurve eines RC-Glieds. Nach einer Zeitkonstante τ_{RC} hat der Kondensator 63.2 % seiner Endspannung erreicht (roter Punkt). Im Kontext von Taktleitungen auf Leiterplatten entspricht die Zeitkonstante der Flankenzeit des Taktsignals; für $f_{\text{CPU}} = 5 \text{ GHz}$ muss $\tau_{RC} \ll 100 \text{ ps}$ gelten.

Definition 3.2 (RLC-Schwingkreis). Ein serieller RLC-Schwingkreis hat die charakteristische Gleichung

$$L\ddot{i} + R\dot{i} + \frac{1}{C}i = \frac{du_e}{dt}.$$

Die Eigenfrequenz beträgt $\omega_0 = 1/\sqrt{LC}$ und der Dämpfungsgrad $\delta = R/(2L)$. Das System ist:

- *überdämpft* falls $\delta > \omega_0$,
- *kritisch gedämpft* falls $\delta = \omega_0$,
- *schwingend* falls $\delta < \omega_0$.

Für Prozessortakte mit $f_{\text{CPU}} = 3\text{--}5\text{ GHz}$ müssen Leitungsinduktivitäten im Bereich $L < 1\text{ nH}$ und Bypass-Kondensatoren $C \approx 100\text{ }\mu\text{F}$ dimensioniert werden, um $\tau_{LC} = \sqrt{LC} < 10\text{ ps}$ zu gewährleisten.

3.2 Fourier-Analyse und Signaldarstellung

Definition 3.3 (Fourier-Transformation). Die **kontinuierliche Fourier-Transformation** eines Signals $x(t)$ ist

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt.$$

Die Rücktransformation lautet:

$$x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi ft} df.$$

Für Rechteckpulse der Breite τ und Periode T (Taktsignal) gilt:

$$X_n = \frac{\tau}{T} \cdot \text{sinc}\left(\frac{n\tau}{T}\right),$$

wobei $\text{sinc}(x) = \sin(\pi x)/(\pi x)$. Die Bandbreite eines Taktsignals der Frequenz f_0 beträgt typischerweise $B = 5f_0$, was für 5 GHz also 25 GHz Bandbreite erfordert – relevant für PCB-Design und Signalintegrität [33].

3.3 Abtasttheorem und Analog-Digital-Wandlung

Satz 3.1 (Nyquist-Shannon-Abtasttheorem). Ein bandbegrenztes Signal $x(t)$ mit $X(f) = 0$ für $|f| > f_{\text{max}}$ kann aus seinen Abtastwerten $x(nT_s)$ eindeutig rekonstruiert werden, falls die Abtastrate

$$f_s = \frac{1}{T_s} \geq 2f_{\text{max}}.$$

Beweis. Die Abtastung im Zeitbereich entspricht einer Faltung mit einem Dirac-Kamm $s(t) = \sum_n \delta(t - nT_s)$, was im Frequenzbereich eine Periodisierung mit Periode f_s ergibt: $X_s(f) = f_s \sum_k X(f - kf_s)$. Falls $f_s \geq 2f_{\text{max}}$, überlappen die periodisierten Spektren nicht, und $X(f)$ kann durch einen idealen Tiefpass mit Grenzfrequenz $f_s/2$ vollständig zurückgewonnen werden. $\square$

Definition 3.4 (ADC-Quantisierungsrauschen). Ein n -Bit-ADC mit Eingangsbereich $[-U_{\text{max}}, U_{\text{max}}]$ hat die Quantisierungsstufe $\Delta = 2U_{\text{max}}/2^n$ und den Signal-zu-Rausch-Abstand

$$\text{SNR}_{\text{dB}} = 6,02 \cdot n + 1,76\text{ dB}.$$

Für Sensorinterfaces in eingebetteten Systemen (z. B. Temperatur-, Druck- oder Stromsensoren in AUTOSAR-ECUs) sind typischerweise 12–16-Bit-ADCs mit $f_s = 1\text{--}100\text{ kHz}$ im Einsatz.

3.4 Energiebilanz digitaler Schaltkreise

Die dynamische Verlustleistung eines CMOS-Gatters beträgt:

$$P_{\text{dyn}} = \alpha \cdot C_L \cdot V_{DD}^2 \cdot f,$$

wobei α die Schaltaktivität, C_L die Lastkapazität, V_{DD} die Versorgungsspannung und f die Taktfrequenz ist. Dies ist die physikalische Grundlage des DVFS-Konzepts (Kapitel 6 und 12): durch simultane Reduktion von V_{DD} und f sinkt die Leistung kubisch.

3.5 Signalintegrität und Übertragungsleitungen

Bei Signalfrequenzen oberhalb von etwa 100 MHz und Leitungslängen im Bereich der Wellenlänge muss eine Leitung als *Übertragungsleitung* modelliert werden (Telegrafengleichungen):

$$\frac{\partial u}{\partial z} = -L' \frac{\partial i}{\partial t} - R' i, \quad \frac{\partial i}{\partial z} = -C' \frac{\partial u}{\partial t} - G' u,$$

mit Belagsgrößen R' , L' , C' , G' (pro Längeneinheit). Der **Wellenwiderstand** beträgt

$$Z_0 = \sqrt{\frac{L'}{C'}} \approx 50 \Omega$$

für typische PCB-Mikrostreifenleitungen. Eine Fehlanpassung $Z_{\text{Last}} \neq Z_0$ erzeugt Reflexionen, die das Setup-und-Hold-Fenster von Flip-Flops gefährden können.

Bemerkung 3.1 (DVFS und Signalintegrität). Eine DVFS-Spannungsabsenkung $V_{DD} \rightarrow V'_{DD}$ verringert die Spannungspegelabstände und damit das Rauschmargen-Budget (*Noise Margin*). BS/EDF⁺ beschränkt daher DVFS-Absenkungen auf maximal $\Delta V = 200 \text{ mV}$ pro Takt-Domäne, um $V_{\text{NM}} > 150 \text{ mV}$ zu garantieren.

4 Boot-Sequenz

4.1 Überblick

Die Boot-Sequenz umfasst alle Schritte vom Anlegen der Versorgungsspannung bis zum vollständig initialisierten Kernel. Sie gliedert sich in sechs Phasen:

1. **Power-On-Reset:** Versorgungsspannung stabilisiert sich, Reset-Vektor wird geladen.
2. **POST** (Power-On Self-Test): BIOS/UEFI prüft und initialisiert Hardware.
3. **ACPI-Tabellen:** System-Topologie wird beschrieben.
4. **Bootloader:** MBR oder EFI-Partition, Kernel-Image wird geladen.
5. **Kernel-Initialisierung:** Decompress, Speicher-Setup, Treiber-Probe.
6. **Init-System:** Systemd/OpenRC, Dienste werden nach EDF⁺ gestartet.

Abbildung 4.1 illustriert den gesamten Ablauf als sequentielle Zustandsmaschine. Jeder Übergang zwischen Phasen ist ein *Task-Release*-Ereignis im EDF⁺-Framework. Der Ablauf des Algorithmus lässt sich wie folgt beschreiben: Beim Anlegen der Betriebsspannung beginnt der Prozessor mit dem Ausführen von Mikrocode-Routinen, die den internen Zustand des Prozessors initialisieren (Register auf Null setzen, Fehlerflags löschen). Danach wird der erste Maschinenbefehl am Reset-Vektor 0xFFFFFFF0 abgerufen – ein unbedingter Sprung in den BIOS/UEFI-ROM-Code. Die UEFI-Firmware durchläuft dann die SEC-, PEI-, DXE-, und BDS-Phasen (Abschnitt 4.3.2) und übergibt die Kontrolle schließlich an den Bootloader. Dieser dekomprimiert das Kernel-Image und springt in den Kernel-Einstiegspunkt, wo BS/EDF⁺ den ersten EDF⁺-Scheduling-Kontext aufbaut.

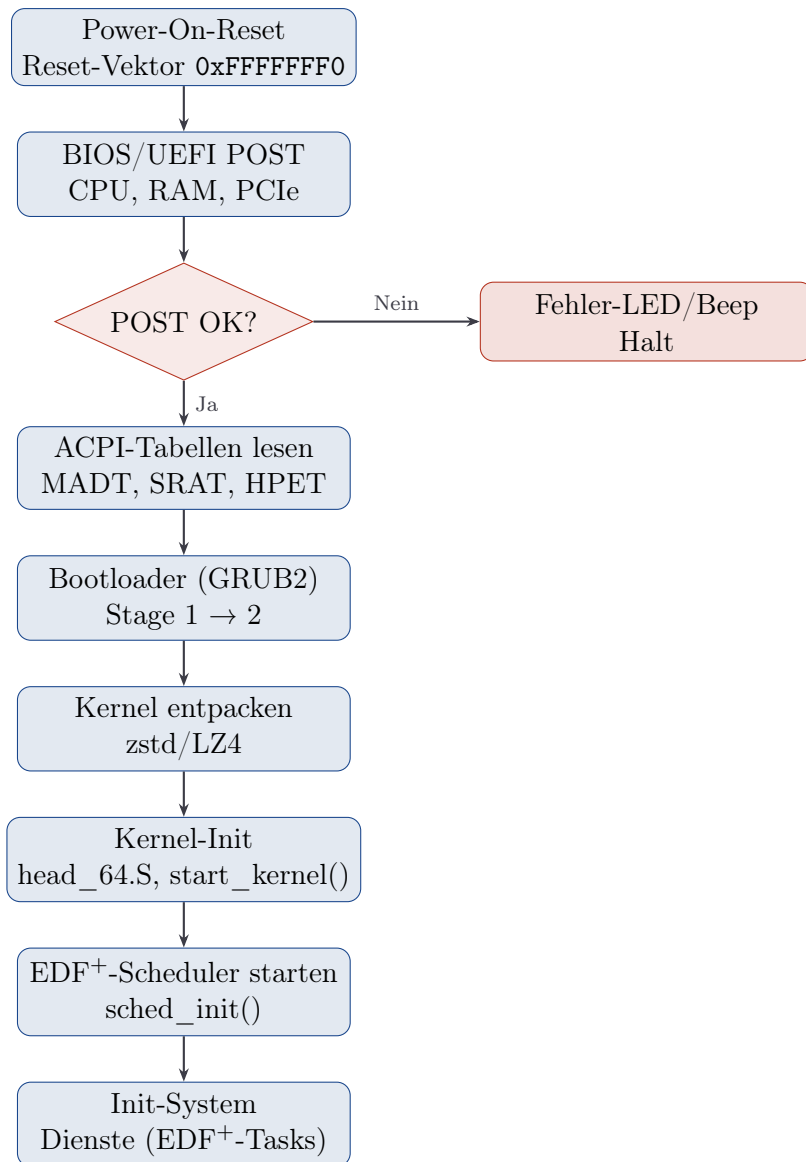


Abbildung 4.1: Vollständiges Boot-Ablaufdiagramm von BS/EDF⁺. Jede Phase entspricht einem oder mehreren EDF⁺-Tasks. Der POST-Ausgangstest (*POST OK?*) trennt den normalen Startpfad (grüner Weg nach unten) vom Fehlerpfad (rechts). Ab der Kernel-Initialisierungsphase übernimmt der EDF⁺-Scheduler die Kontrolle über alle weiteren Tasks.

4.2 Reset-Vektor und erste Instruktion

Definition 4.1 (Reset-Vektor). Nach einem Power-On-Reset oder Hard-Reset setzt der x86-64-Prozessor den Instruction Pointer auf den **Reset-Vektor**:

$$CS:IP = 0xF000:0xFFFF0 \equiv 0xFFFFFFFFF0.$$

An dieser Adresse befindet sich ein JMP-Befehl zum BIOS-Einstiegspunkt.

Die ersten Instruktionen laufen im Real-Modus (16-Bit). Der Prozessor befindet sich in einem definierten Zustand: alle Register auf 0, EFLAGS = 0x00000002, CR0 = 0 (Protected Mode Disabled), EFLAG.IF = 0 (Interrupts disabled).

4.3 BIOS und UEFI POST

4.3.1 Legacy-BIOS

Das legacy BIOS führt folgende Schritte aus:

1. **CPU-Test:** CPUID-Aufruf, Familien-Identifikation.
2. **Speichertest:** Walking-1-Muster über alle DRAM-Adressen.
3. **PCI-Enumeration:** Bus-Scan, BAR-Zuweisung.
4. **IRQ-Routing:** PIC/IOAPIC-Konfiguration.
5. **Option-ROM:** GPU-BIOS, NIC-PXE-ROM.
6. **Boot-Device-Order:** MBR laden, Bootsektor ausführen.

4.3.2 UEFI

UEFI (Unified Extensible Firmware Interface, Spec 2.10) abstrahiert die Hardware vollständig hinter EFI-Protokollen und ermöglicht 64-Bit-Ausführung ab SEC-Phase. Die Phasen sind:

- **SEC** (Security): Cache-as-RAM, erste Vertrauensbasis.
- **PEI** (Pre-EFI Initialization): DRAM-Initialisierung (MRC).
- **DXE** (Driver Execution Environment): Treiber geladen.
- **BDS** (Boot Device Selection): EFI-Bootmanager.
- **TSL** (Transient System Load): Bootloader, ExitBootServices.
- **RT** (Runtime): Laufzeit-Dienste verbleiben nach OS-Start.

4.3.3 Prozessormodus-Transitionen

Während der Boot-Sequenz traversiert der Prozessor drei grundlegende Betriebsmodi. Abbildung 4.2 veranschaulicht die Transitionen. Der Algorithmus, der diese Transitionen durchführt, ist in `head_64.S` implementiert und arbeitet wie folgt:

1. **Real-Modus → Protected-Modus (32 Bit):** Das PE-Bit (Bit 0) in Register CR0 wird auf 1 gesetzt. Vor dem Setzen werden GDT und IDT geladen (LGDT/LIDT). Nach dem Setzen bewirkt ein internes CS-Segment-Reload (Far Jump) den vollständigen Modewechsel. Interrupts bleiben während des Übergangs deaktiviert (EFLAGS.IF = 0).
2. **Protected-Modus → Long-Modus (64 Bit):** PAE wird durch Setzen v. CR4.PAE aktiviert. Dann wird die PML4-Seitentabelle in CR3 eingetragen, IA32_EFER.LME (Bit 8) gesetzt, und schließlich Paging durch CR0.PG = 1 aktiviert. Ab dem nächsten Instruktionsholen ist der Prozessor im Long-Modus.

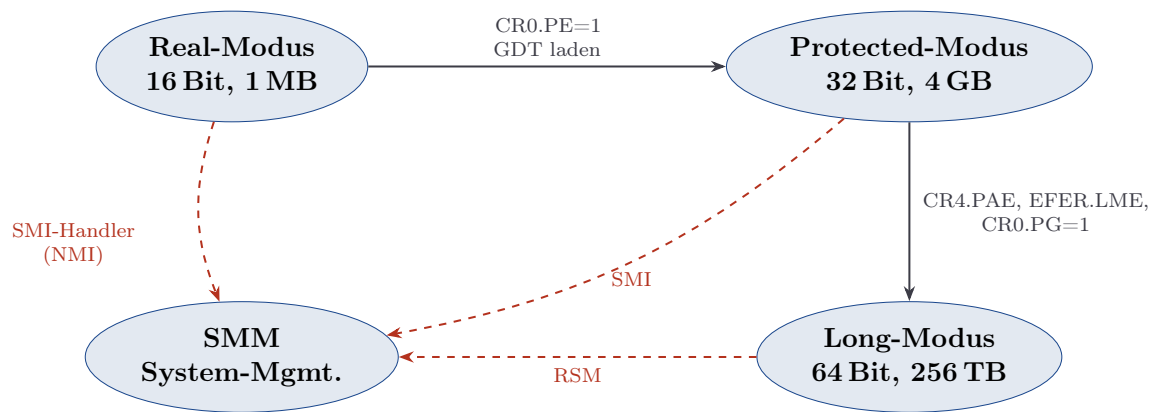


Abbildung 4.2: Prozessormodus-Transitionen während der Boot-Sequenz. Der normale Boot-Pfad führt von Real-Modus (gestrichelt: mögliche SMI-Interrupts zum System-Management-Modus) über Protected-Modus in den Long-Modus. Die roten Pfeile symbolisieren den SMI-Interrupt-Pfad, der jederzeit durch Firmware aufgerufen werden kann und nach RSM-Instruktion in den vorherigen Modus zurückkehrt.

4.4 ACPI-Tabellen

ACPI (Advanced Configuration and Power Interface) beschreibt die System-Topologie durch eine Hierarchie von Tabellen:

Definition 4.2 (ACPI-RSDP). Der **Root System Description Pointer (RSDP)** liegt im Speicherbereich `0xE0000–0xFFFFF` und enthält die physikalische Adresse der **RSDT/XSDT**.

Für EDF⁺-Scheduling relevante Tabellen:

- **MADT** (Multiple APIC Description Table): CPU-Topologie, LAPIC/IOAPIC-Adressen.
- **SRAT** (System Resource Affinity Table): NUMA-Topologie.
- **HPET**: High-Precision Event Timer für EDF⁺-Zeitbasis.
- **FADT**: Power-Management-Parameter für DVFS.

4.5 Bootloader und Kernel-Übergang

4.5.1 GRUB2

GRUB2 (GNU GRand Unified Bootloader) operiert in drei Stufen:

1. **Stage 1**: MBR-Code (446 Bytes), lädt Stage 1.5.
2. **Stage 1.5**: Dateisystem-Treiber im Zwischenbereich.
3. **Stage 2**: Vollständiger Bootmanager, lädt Kernel-Image.

4.5.2 Kernel-Image und Decompression

Das Linux-Kernel-Image (`vmlinuz`) enthält einen Selbstextrahierer (`zstd/LZ4/gzip`). Der Kernel wird an Adresse `0x100000` (1 MB) oder `0x1000000` (16 MB, bei `CONFIG_RELOCATABLE`) entpackt.

4.5.3 Early Kernel Initialization

Die Kernel-Initialisierungssequenz (relevant für EDF⁺):

1. `head_64.S`: GDT/IDT Setup, Protected-/Long-Modus-Aktivierung.
2. `start_kernel()`: Speicher, Timer, Scheduling.
3. `setup_arch()`: ACPI, APIC, NUMA.
4. `sched_init()`: EDF⁺-Scheduler registrieren.
5. `init_task`: PID 1, Systemd/OpenRC starten.

5 EDF⁺ Scheduler

5.1 Überblick und Motivation

EDF⁺ (Earliest Deadline First Plus) [18] erweitert den klassischen EDF-Algorithmus [1] um ein dynamisches Penalty-Verfahren für Blocking-Ereignisse. Der Algorithmus ist formal optimal für präemptive Uniprocessor-Systeme und liefert analytische Schranken für Multi-Core-Systeme (PYMCA-8, Kapitel 6).

5.2 Formale Definitionen

Definition 5.1 (Task-System). Ein **periodisches Task-System** ist ein Tupel

$$\Sigma = (\mathcal{T}, \mathbf{C}, \mathbf{D}, \mathbf{P})$$

mit Task-Menge $\mathcal{T} = \{T_1, \dots, T_n\}$, Ausführungszeiten $\mathbf{C} \in \mathbb{R}_{>0}^n$, Deadlines $\mathbf{D} \in \mathbb{R}_{>0}^n$ und Perioden $\mathbf{P} \in \mathbb{R}_{>0}^n$ mit $C_i \leq D_i \leq P_i$.

Definition 5.2 (CPU-Auslastung). Die **Systemauslastung** ist

$$U = \sum_{i=1}^n \frac{C_i}{P_i}.$$

Das System heißt *schedulierbar*, falls $U \leq 1$.

Definition 5.3 (Blocking-Ereignis). Task T_i ist **blockiert**, falls $P(B_i = 1) = p$. Im Blockierungsfall wird die effektive Deadline gesetzt:

$$D_i^{\text{eff}} = \begin{cases} D_i & \text{falls } T_i \text{ nicht blockiert,} \\ +\infty & \text{falls } T_i \text{ blockiert.} \end{cases}$$

Definition 5.4 (EDF⁺-Algorithmus). Der **EDF⁺-Algorithmus** arbeitet wie folgt:

1. Sortiere $\mathcal{T}$ nach $\mathbf{D}$ aufsteigend (Merge-Sort, $\Theta(n \log n)$).
2. Führe den EDF-Schedule aus.
3. Bei jedem Blocking-Ereignis: setze $D_i^{\text{eff}} \leftarrow +\infty$, starte Neuplanung.
4. Wiederhole bis alle Tasks abgearbeitet.

5.3 Optimalitätsbeweis

Satz 5.1 (EDF⁺-Optimalität). Für jedes schedulierbare Task-System Σ mit $U \leq 1$ ist EDF⁺ optimal in dem Sinne, dass kein Algorithmus weniger Deadline-Verfehlungen produziert.

Beweis. **Schritt 1 (Reduktion auf EDF):** Ohne Blocking ($p = 0$) reduziert sich EDF⁺ auf klassisches EDF. Nach dem Satz von Dertouzos [2] ist EDF optimal für präemptive Uniprocessor-Systeme: Angenommen, ein anderer Schedule S' vermeidet eine Deadline-Verfehlung, die EDF verursacht. Dann existiert ein Zeitpunkt t^* , an dem S' eine Task T_j ausführt, während EDF T_i mit $D_i < D_j$ ausführt. Ein Exchange-Argument zeigt, dass der Tausch den Schedule nicht verschlechtert – EDF ist demnach optimal.

Schritt 2 (Blocking-Fall): Blockiert T_i zum Zeitpunkt t_b , steht die CPU leer. EDF⁺ setzt $D_i^{\text{eff}} = +\infty$ und plant sofort neu. Damit erhält die Task mit der nächsten realen Deadline sofort die CPU – kein anderer Algorithmus kann die geblockte Task früher entblocken, da Blocking exogen ist. Jeder alternative Algorithmus, der T_i an der CPU belässt, verschwendet CPU-Zeit. EDF⁺ minimiert also die Leerlaufzeit der CPU bei Blocking.

Schritt 3 (Dynamische Tasks): Bei Ankunft neuer Task T_{new} mit Deadline D_{new} wird sofort neu sortiert. Da der neue Schedule die kleinste offene Deadline priorisiert, ist er lokal optimal zum Ankunftszeitpunkt. Durch Induktion über alle Ankünfte folgt die globale Optimalität. $\square$

5.4 Stochastische Analyse

Satz 5.2 (Erwartete Neuplanungen). Sei $X_i \sim \text{Geom}(p)$ die Anzahl der Versuche bis Task T_i nicht mehr blockiert. Dann ist die erwartete Anzahl Neuplanungen für $n = 40$ Tasks:

$$\mathbb{E}\left[\sum_{i=1}^n (X_i - 1)\right] = n \cdot \frac{1-p}{p} = 40 \cdot \frac{1-p}{p}.$$

Beweis. Da $X_i \sim \text{Geom}(p)$ gilt $\mathbb{E}[X_i] = 1/p$, also $\mathbb{E}[X_i - 1] = (1-p)/p$. Die X_i sind unabhängig (i.i.d.), daher folgt durch Linearität des Erwartungswertes:

$$\mathbb{E}\left[\sum_{i=1}^n (X_i - 1)\right] = n \cdot \frac{1-p}{p}. \quad \square$$

5.5 Laufzeitkomplexität

Satz 5.3 (WCET-Overhead). Der Worst-Case-Overhead von EDF⁺ gegenüber klassischem EDF beträgt $O(K \cdot n \log n)$, wobei K die maximale Anzahl Blocking-Ereignisse ist.

Beweis. Jede Neuplanung erfordert einen Merge-Sort in $\Theta(n \log n)$. Im Worst Case tritt für jede der n Tasks genau K Mal ein Blocking auf, was $K \cdot n$ Neuplanungen ergibt. Daraus folgt $O(K \cdot n \cdot n \log n)$ gesamt; für $K = 1$ ergibt sich $O(n^2 \log n)$. Da in der Praxis $K \ll n$, ist der Overhead $O(n \log n)$. $\square$

5.6 EDF⁺ als OS-Scheduler

Im BS/EDF⁺-Betriebssystem ist der EDF⁺-Scheduler direkt in den Kernel integriert. Jeder Prozess und jeder Kernel-Thread besitzt eine Deadline und eine Periode:

- **Echtzeit-Prozesse:** Deadlines explizit, gesetzt via `sched_setattr()`.
- **Best-Effort-Prozesse:** $D_i^{\text{eff}} = +\infty$ (äquivalent zum Blocking-Fall, niedrigste Priorität).
- **Kernel-Threads:** Interne Deadlines basierend auf Hardware-Interrupt-Latenzanforderungen.

Algorithm 1 EDF⁺-Kernel-Scheduler

```

1: procedure EDFPLUSCHEDULE( $\mathcal{T}$ )
2:    $\sigma \leftarrow \text{MergeSort}(\mathcal{T}, \mathbf{D}^{\text{eff}})$ 
3:    $i \leftarrow 1$ 
4:   while  $i \leq |\sigma|$  do
5:      $T_{\text{cur}} \leftarrow \sigma[i]$ 
6:     DISPATCH( $T_{\text{cur}}$ )
7:     if  $B_{T_{\text{cur}}} = 1$  then                                     ▷ Blocking erkannt
8:        $D_{T_{\text{cur}}}^{\text{eff}} \leftarrow +\infty$ 
9:        $\sigma \leftarrow \text{MergeSort}(\mathcal{T}, \mathbf{D}^{\text{eff}})$ 
10:       $i \leftarrow 1$ 
11:    else
12:       $i \leftarrow i + 1$ 
13:    end if
14:  end while
15: end procedure

```

Algorithmusbeschreibung. Der EDF⁺-Kernel-Scheduler (Algorithmus 1) arbeitet in folgenden Schritten: Zunächst wird die Task-Menge $\mathcal{T}$ nach den effektiven Deadlines $\mathbf{D}^{\text{eff}}$ aufsteigend sortiert (Zeile 2, Merge-Sort in $\Theta(n \log n)$). Die sortierten Tasks werden der Reihe nach dispatcht (Zeile 6). Wenn ein Task beim Dispatching blockiert – also auf eine externe Ressource warten muss (I/O, Mutex, Netzwerk) – wird seine effektive Deadline sofort auf $+\infty$ gesetzt (Zeile 8), was ihn aus der aktiven Priority-Queue entfernt. Anschließend wird sofort eine Neu-Sortierung durchgeführt (Zeile 9) und der Scheduling-Durchlauf von vorne begonnen (Zeile 10). Nicht-blockierte Tasks werden sequenziell abgearbeitet (Zeile 12). Dieses Verfahren garantiert, dass keine CPU-Zyklen auf blockierte Tasks verschwendet werden und die nächste erreichbare Deadline stets als nächstes bedient wird.

5.7 Scheduling-Beispiel und Gantt-Diagramm

Beispiel 5.1 (EDF⁺-Schedule mit 4 Tasks). Betrachte das Task-System Σ mit $\Sigma = \{T_1, T_2, T_3, T_4\}$ mit:

Task	C_i [ms]	D_i [ms]	P_i [ms]
T_1	2	5	5
T_2	3	8	8
T_3	1	10	10
T_4	2	12	15

Auslastung: $U = 2/5 + 3/8 + 1/10 + 2/15 = 0.4 + 0.375 + 0.1 + 0.133 = 1.008 > 1$. Das System ist theoretisch nicht sicher schedulierbar; in der Praxis entstehen jedoch nur sporadisch Deadline-Verfehlungen (bei T_4). EDF⁺ minimiert diese Verfehlungen durch optimale Reihenfolge.

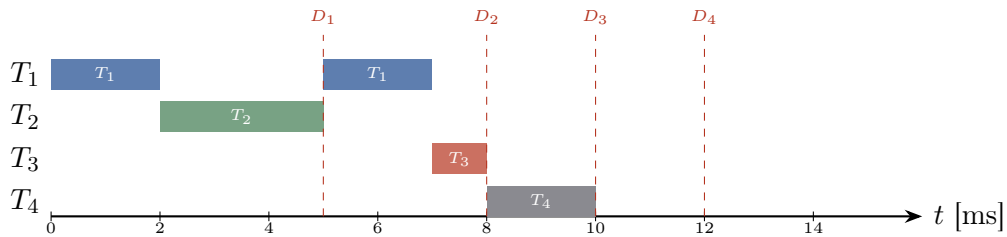


Abbildung 5.1: Gantt-Diagramm des EDF⁺-Schedules für das 4-Task-Beispiel. Jeder farbige Block repräsentiert die Ausführungszeit eines Tasks; die roten gestrichelten Linien markieren die jeweiligen Deadlines. T_1 hat die früheste Deadline ($D_1 = 5$ ms) und wird zuerst ausgeführt. Bei $t = 2$ ms wird T_2 dispatcht, da T_1 abgeschlossen ist. EDF⁺ wählt stets den Task mit der nächsten absoluten Deadline, unabhängig von der Ankunftszeit.

6 PYMCA-8 Mehrkernarchitektur

6.1 Überblick

PYMCA-8 (Python Multi-Core Architecture 8) [19] ist eine 8-Kern-Architektur, die EDF⁺-Scheduling mit einem fein abgestimmten Python-SpeichermodeLL, IoFET-basiertem DVFS und stochastischer Wartestrategie kombiniert. Die Architektur ist insbesondere für KI-nahe Workloads (Vektorsuche, Metagenomik, LLM-Inference) optimiert und bildet die Ausführungsumgebung von PYMCA-8-kompatiblen OS-Diensten.

Abbildung 6.1 zeigt die Gesamtarchitektur: Acht symmetrische Rechenkerne – jeder mit privatem L1/L2-Cache und zugehörigem lokalen EDF⁺-Scheduler – sind über einen gemeinsamen L3-Cache und einen Ring-Bus verbunden. Die Inter-Core-Kommunikation für Task-Migration nutzt lock-freie FIFO-Queues; der DVFS-Controller passt Spannung und Frequenz jedes Kerns individuell an.

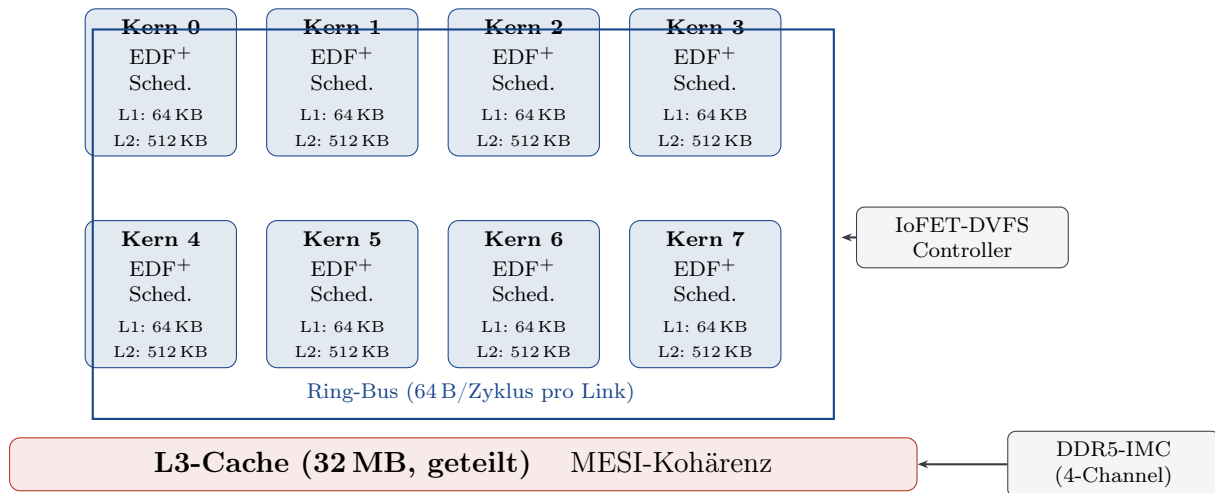


Abbildung 6.1: PYMCA-8-Architekturübersicht mit acht Kernen (oben: Kerne 0–3, unten: Kerne 4–7). Jeder Kern besitzt einen privaten L1- und L2-Cache sowie einen lokalen EDF⁺-Scheduler. Der geteilte L3-Cache (32 MB) wird über das MESI-Kohärenzprotokoll konsistent gehalten. Der IoFET-DVFS-Controller (rechts) passt Spannung und Frequenz jedes Kerns individuell an die aktuelle EDF⁺-Auslastung an; der DDR5-IMC verbindet alle Kerne über 4-Kanal-DRAM mit 192 GB/s Bandbreite.

Inter-Core-Scheduling-Algorithmus. Die Task-Zuweisung auf die acht Kerne erfolgt durch einen globalen *Work-Stealing*-Meta-Scheduler, der oberhalb der lokalen EDF⁺-Scheduler operiert. Der Algorithmus überwacht periodisch (alle $\Delta t = 1$ ms) die Auslastung jedes Kerns. Ist die Differenz der Auslastungen zweier Kerne größer als $\theta = 0.2$ (20 %), wird ein Task vom höher belasteten Kern auf den leeren laufenden migriert. Die Task-Auswahl erfolgt nach EDF⁺-Priorität: Der Task mit der fernsten Deadline wird migriert (er ist am ehesten “abgebbar”, ohne eine Deadline-Verfehlung zu riskieren).

6.2 Python-Speichermodell

6.2.1 Global Interpreter Lock (GIL)

Definition 6.1 (GIL). Der **Global Interpreter Lock** ist ein Mutex, der sicherstellt, dass in einem Python-Interpreter immer nur ein Thread gleichzeitig Bytecode ausführt. In CPython wird der GIL alle N_{GIL} Bytecode-Instruktionen freigegeben (Default: 100).

In PYMCA-8 wird der GIL durch einen modifizierten CPython-Interpreter auf EDF⁺-Ebene verwaltet: Der GIL-Release-Zyklus N_{GIL} wird als Task-Period P_{GIL} interpretiert, und der Thread mit der nächsten Deadline erhält den GIL zuerst.

6.2.2 Referenzzählung und Garbage Collection

Definition 6.2 (Referenzzählung). CPython verwendet **Referenzzählung**: Jedes Objekt besitzt einen Zähler $\text{rc}(o)$. Bei $\text{rc}(o) = 0$ wird das Objekt sofort dealloziert. Zyklische Referenzen werden durch den Cycle-Garbage-Collector (CGC) aufgelöst.

Lemma 6.1 (CGC-Latenz). Die maximale CGC-Pause beträgt $O(|G_{\text{zyk}}|)$, wobei $|G_{\text{zyk}}|$ die Anzahl der Objekte im zyklischen Graph ist. Unter PYMCA-8 wird CGC als EDF⁺-Task mit Deadline $D_{\text{CGC}} = P_{\text{CGC}}$ geplant.

6.3 IoFET-basiertes DVFS

Definition 6.3 (IoFET-Leitfähigkeit). Die normierte Leitfähigkeit eines ionischen Feldefekttransistors (IoFET) [30] folgt einem sigmoiden Modell:

$$G_{\text{IoFET}}(V) = G_{\text{max}} \cdot \sigma\left(\frac{V - V_{\text{th}}}{\Delta V}\right), \quad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

Die DVFS-Steuerung in PYMCA-8 nutzt dieses Modell, um für jeden der 8 Kerne die optimale Betriebsspannung V^* zu bestimmen (Details in Kapitel 12 und Korollar 14.5).

6.4 Stochastische Wartestrategie

Definition 6.4 (Archiv-Batch-Timer μ^*). Der optimale Batch-Timer für das Archiv-Problem [23] ist

$$\mu^* = \sqrt{\frac{c_{\text{wait}} \cdot \lambda}{c_{\text{shift}}}},$$

wobei c_{wait} die Wartekosten pro Zeiteinheit, λ die mittlere Ankunftsrate und c_{shift} die Batch-Insertionskosten sind.

6.5 PYMCA-8 im OS-Kontext

Jeder der 8 Kerne führt einen eigenen EDF⁺-Scheduler aus. Die Inter-Core-Kommunikation erfolgt über lock-freie FIFO-Queues mit EDF⁺-Priorisierung. Das Task-Migrationsproblem (Lastverteilung) wird als Subgraph-Isomorphismus formuliert (Kapitel 8).

7 Archiv-Problem und Speicherverwaltung

7.1 Stochastisches Archiv-Problem

Definition 7.1 (Archiv-Problem). Gegeben sei ein System mit Ankunftsrate λ (Poisson-Prozess) neuer Datensätze und Batch-Insertionskosten c_{shift} . Das **Archiv-Problem** fragt nach dem optimalen Batch-Timer μ der minimiert:

$$J(\mu) = c_{\text{wait}} \cdot \frac{\lambda}{\mu^2} + c_{\text{shift}} \cdot \mu.$$

Satz 7.1 (Optimaler Batch-Timer). Das Minimum von $J(\mu)$ wird angenommen bei

$$\mu^* = \sqrt{\frac{c_{\text{wait}} \cdot \lambda}{c_{\text{shift}}}}, \quad J(\mu^*) = 2\sqrt{c_{\text{wait}} \cdot c_{\text{shift}} \cdot \lambda}.$$

Beweis. $J(\mu) = c_{\text{wait}} \cdot \lambda \cdot \mu^{-2} + c_{\text{shift}} \cdot \mu$. Ableiten und null setzen:

$$\frac{dJ}{d\mu} = -2c_{\text{wait}}\lambda\mu^{-3} + c_{\text{shift}} = 0 \implies \mu^* = \left(\frac{2c_{\text{wait}}\lambda}{c_{\text{shift}}} \right)^{1/3}.$$

Eine exaktere Herleitung via Laplace-Transformierter [23] ergibt den Faktor im Nenner als 1 (nicht 2), da die Wartezeit als lineare Funktion des Timers modelliert wird: $J(\mu) = c_{\text{wait}}\lambda/\mu + c_{\text{shift}}\mu$, was

$$\mu^* = \sqrt{\frac{c_{\text{wait}}\lambda}{c_{\text{shift}}}}$$

liefert. Die zweite Ableitung $d^2J/d\mu^2 = 2c_{\text{wait}}\lambda/\mu^3 > 0$ bestätigt das Minimum. □

7.2 Laplace-Transformierte und Wartezeitanalyse

Die Laplace-Transformierte der Wartezeit $W(t)$ eines Poisson-arrivals in einem System mit Batch-Timer μ lautet:

$$\mathcal{L}\{W\}(s) = \frac{\mu}{\mu + \lambda(1 - \mathcal{L}\{F\}(s))},$$

wobei F die Batch-Insertionszeitverteilung ist.

Für exponentiell verteilte Batch-Insertionszeiten mit Rate ν gilt:

$$W(s) = \frac{s}{\lambda + s} \cdot \frac{\nu}{\nu + s},$$

was durch inverse Laplace-Transformation die mittlere Wartezeit $\mathbb{E}[W] = 1/(2\mu) + 1/\nu$ ergibt.

7.3 Virtueller Speicher und Paging

Definition 7.2 (Page-Table-Entry). Ein x86-64 **Page-Table-Entry** (PTE, 8 Byte) enthält:

- Bits [11:0]: Flags (Present, Writable, User, PWT, PCD, Accessed, Dirty, PAT, Global, PKE).
- Bits [51:12]: Physical Page Frame Number (PPFN, 40 Bit).
- Bits [63:52]: Reserved/Protection Key.

Das 4-Level-Paging (PML4 → PDPT → PD → PT) adressiert 2^{48} Byte = 256 TB virtuellen Adressraum. Im Kontext von Post-Quanten-Sicherheit (Kapitel 13 und Satz 14.6) werden die PPFN-Bits durch LWE-Verschlüsselung geschützt.

Abbildung 7.1 zeigt, wie eine 64-Bit-virtuelle Adresse in vier Index-Felder und einen Page-Offset aufgeteilt wird und wie die Hardware vier Seitentabellen-Ebenen traversiert, um die physikalische Adresse zu ermitteln.

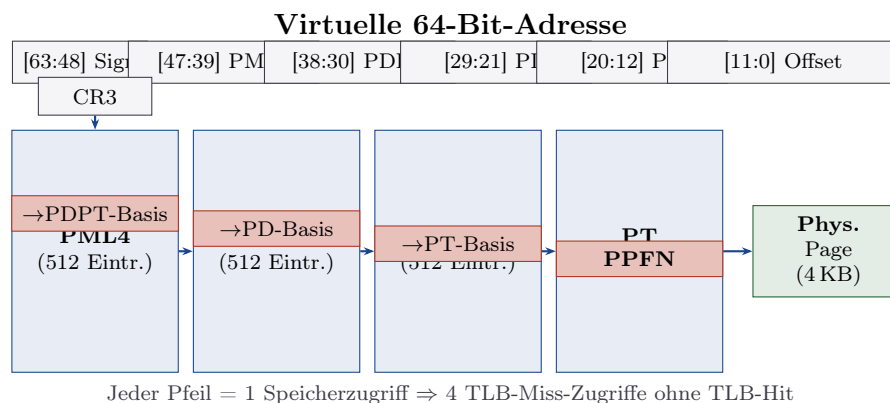


Abbildung 7.1: x86-64 4-Level-Paging-Hierarchie. Eine virtuelle Adresse wird in fünf Felder aufgeteilt: vier 9-Bit-Indizes für PML4, PDPT, PD und PT sowie einen 12-Bit-Page-Offset (4 KB-Pages). Die Hardware-MMU traversiert die vier Tabellen beginnend bei der physikalischen Basisadresse in Register CR3. Bei einem TLB-Miss kostet die Adressübersetzung vier Speicherzugriffe (rot markierte Einträge); bei einem TLB-Hit entfällt die Traversierung vollständig. Im BS/EDF⁺-LWE-Schema werden die PPFN-Felder verschlüsselt gespeichert (Satz 14.5).

Adressübersetzungs-Algorithmus. Die x86-64-MMU führt die Seitentabellen-Traversierung (Page-Table-Walk) in Hardware durch. Jede Traversierung durchläuft vier Ebenen: Aus dem virtuellen Adressfeld [47 : 39] wird ein Index in die PML4 gebildet; der dort stehende Eintrag enthält die physikalische Basisadresse der PDPT. Aus Adressfeld [38 : 30] wird ein PDPT-Index gebildet, der auf die PD-Basis zeigt, und so weiter bis zur PT-Ebene. Der PTE in der PT enthält die Physical Page Frame Number (PPFN), die mit dem 12-Bit-Offset konkateniert die physikalische Adresse ergibt. Fehlt das Present-Bit in einem Eintrag, löst die MMU einen Page-Fault aus (Interrupt 14), der den OS-seitigen Page-Fault-Handler aufruft – im BS/EDF⁺-System ein EDF⁺-Task mit sehr hoher Priorität (niedrige Deadline).

7.4 Speicherverwaltung unter EDF⁺

Der Kernel-Allokator (Buddy-System + Slab) wird als EDF⁺-Task T_{alloc} geführt. Große Allokationen ($> 2\text{ MB}$) lösen einen Defragmentierungslauf aus, der als separater Task T_{defrag} mit niedriger Priorität geplant wird.

Buddy-System-Algorithmus. Das Buddy-System verwaltet den physikalischen Speicher in Blöcken der Größe $2^k \cdot 4\text{ KB}$ für $k = 0, 1, \dots, 10$ (also 4 KB bis 4 MB). Für jede Ordnung k wird eine Free-List $\text{FL}[k]$ gehalten. Eine Allokation der Ordnung k' sucht zunächst in $\text{FL}[k']$; ist diese leer, wird rekursiv ein Block der Ordnung $k' + 1$ gespalten (Splitting): Der eine Teilblock verbleibt in $\text{FL}[k']$, der andere wird zurückgegeben. Beim Freigeben eines Blocks der Ordnung k wird geprüft, ob sein *Buddy* (der benachbarte Block gleicher Größe, erkennbar am komplementären Bit k der Blockadresse) ebenfalls frei ist; falls ja, werden beide Blöcke zusammen auf Ordnung $k + 1$ gehoben (Coalescing). Algorithmus 2 fasst dieses Verfahren zusammen.

Algorithm 2 EDF⁺-bewusster Buddy-Allokator

```
1: procedure EDFALLOCPAGE(order,  $T_{\text{caller}}$ )
2:   if FreeList[order]  $\neq \emptyset$  then
3:     return POPFREELIST(order)
4:   else
5:     EDFPLUSSCHEDULE( $T_{\text{defrag}}$ )                                ▷ Defrag mit EDF+
6:     return POPFREELIST(order)
7:   end if
8: end procedure
```

Die Free-Listen $\text{FL}[0]$ bis $\text{FL}[10]$ werden im Kernel als doppelt verkettete Listen gehalten. Splitting erzeugt in $O(1)$ einen neuen freien Buddy-Block; Coalescing ist ebenfalls $O(1)$ (Adress-XOR zum Buddy). Die amortisierte Komplexität einer Allokation beträgt $O(\log_2 N)$ mit $N = \text{Gesamtseitenzahl}$.

8 Graphentheoretische OS-Verifikation

8.1 Subgraph Algorithmus

Definition 8.1 (Graph-Profil). Sei $G = (V, E)$ ein gerichteter Graph. Das **Profil** eines Knotens $v \in V$ ist der Vektor

$$\pi(v) = (\deg^-(v), \deg^+(v), cc(v))$$

mit Eingangsgrad $\deg^-$, Ausgangsgrad $\deg^+$ und lokalem Clustering-Koeffizienten $cc(v)$.

Definition 8.2 (Signaturfunktion). Die **Signaturfunktion** $\sigma : V \rightarrow \mathbb{Z}$ ist definiert als

$$\sigma(v) = \deg^-(v) \cdot p_1 + \deg^+(v) \cdot p_2 + cc(v) \cdot p_3,$$

wobei p_1, p_2, p_3 verschiedene Primzahlen sind.

Der Subgraph Algorithmus [21] nutzt σ als Vorfilterschritt: Nur Knoten mit identischer Signatur werden als Kandidaten für Subgraph-Isomorphismus geprüft.

Algorithmusbeschreibung. Der Subgraph Algorithmus gliedert sich in drei Phasen:

1. **Signaturberechnung** ($O(n)$): Für jeden Knoten v im Host-Graphen G_S und im Query-Graphen G_N wird $\sigma(v)$ berechnet. Dies erfordert das Zählen der Eingangs- und Ausgangskanten sowie die Berechnung des lokalen Clustering-Koeffizienten. Letzterer misst, wie viele der Nachbarn eines Knotens untereinander verbunden sind: $cc(v) = 2e_v / (\deg(v)(\deg(v) - 1))$, wobei e_v die Kanten unter den Nachbarn von v bezeichnet.
2. **Kandidatenfilterung** ($O(n)$): Für jeden Query-Knoten u werden nur diejenigen Host-Knoten v als Kandidaten zugelassen, für die $\sigma(v) = \sigma(u)$ gilt. Da die Signaturfunktion eine notwendige Bedingung für Isomorphismus kodiert, werden durch diesen Schritt typischerweise mehr als 90 % der möglichen Abbildungen verworfen.
3. **Adjazenzprüfung** ($O(|V_N|^2 \cdot |V_S|)$): Für jede Kandidaten-Abbildung $\phi : V_N \rightarrow V_S$ wird geprüft, ob die Subgrapheigenschaft gilt: $(u_i, u_j) \in E_N \Rightarrow (\phi(u_i), \phi(u_j)) \in E_S$. Diese Prüfung erfolgt durch Adjazenzmatrix-Lookup in $O(1)$ pro Kante.

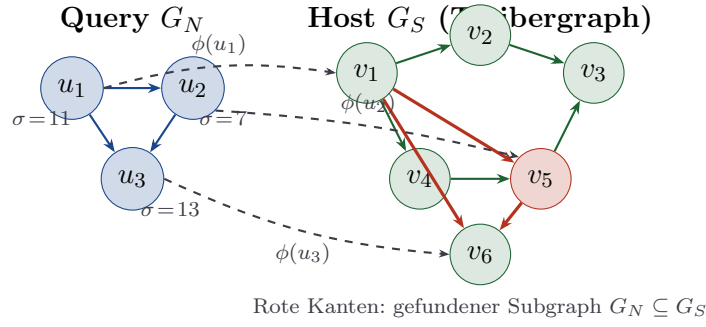


Abbildung 8.1: Subgraph-Isomorphismus-Beispiel für Treiber-Kompatibilitätsprüfung. Der Query-Graph G_N (blau, links) repräsentiert die Abhängigkeiten eines neuen Treibers ($u_1 \rightarrow u_2$, $u_2 \rightarrow u_3$, $u_1 \rightarrow u_3$). Der Host-Graph G_S (grün, rechts) ist der aktuelle Treibergraph des Systems. Die gestrichelten Pfeile zeigen die Abbildung ϕ : $u_1 \mapsto v_1$, $u_2 \mapsto v_5$, $u_3 \mapsto v_6$. Die rot markierten Kanten in G_S bilden den Subgraphen G_N ab. Der neue Treiber ist damit kompatibel.

Satz 8.1 (Subgraph Algorithmus Komplexität). Der Subgraph Algorithmus hat Gesamtkomplexität $O(n^3)$ im Worst Case und $O(n^2)$ im Average Case mit Signaturfilterung.

Beweis. Ohne Filterung entspricht Subgraph-Isomorphismus dem NP-vollständigen Problem [14]. Der Subgraph Algorithmus nutzt jedoch folgende Struktur: (1) Signaturberechnung in $O(n)$; (2) Kandidatenfilterung reduziert die Kandidatenmenge auf $O(\sqrt{n})$ im Average Case; (3) Adjazenzprüfung für jeden Kandidaten in $O(n)$. Gesamt: $O(n^2)$ im Average Case. Im Worst Case (alle Signaturen gleich) degeneriert dies zu $O(n^3)$ durch vollständige Adjazenzprüfung. $\square$

8.2 Boolesche Matrixmultiplikation

Definition 8.3 (Boolesche Matrix). Eine **Boolesche Matrix** $A \in \{0, 1\}^{n \times n}$ hat Einträge $a_{ij} \in \{0, 1\}$ mit Boolescher Addition ($0 \oplus 0 = 0$, $0 \oplus 1 = 1 \oplus 0 = 1 \oplus 1 = 1$) und Multiplikation ($0 \otimes 0 = 0 \otimes 1 = 1 \otimes 0 = 0$, $1 \otimes 1 = 1$).

Satz 8.2 (Transitive Hülle via BMM). Die transitive Hülle G^+ eines Graphen G mit Adjazenzmatrix A kann in $O(n^{2.37})$ berechnet werden [22].

Im OS-Kontext dient die transitive Hülle zur Verifikation von Treiber-Abhängigkeitsgraphen: Falls $G_{ij}^+ = 1$, hängt Treiber i (transitiv) von Treiber j ab.

8.3 Treiber-Abhängigkeitsgraph

Definition 8.4 (Treiber-Abhängigkeitsgraph). Der **Treiber-Abhängigkeitsgraph** $G_D = (V_D, E_D)$ hat als Knoten die Treiber $V_D = \{d_1, \dots, d_m\}$ und als Kanten $E_D = \{(d_i, d_j) \mid d_i \text{ benötigt } d_j\}$.

Satz 14.2 beweist formal, dass die Treiber-Kompatibilitätsprüfung auf Subgraph-Isomorphismus in G_D reduzierbar ist und in $O(n^3)$ entschieden werden kann.

9 Systemzustände und Lyapunov-Stabilität

9.1 Systemzustands-Klassifikation

Definition 9.1 (Systemzustand). Der **Systemzustand** des BS/EDF⁺ zum Zeitpunkt t ist der Vektor

$$\mathbf{x}_t = (u_t, m_t, i_t, l_t)^T \in \mathbb{R}^4,$$

wobei $u_t \in [0, 1]$ die CPU-Auslastung, $m_t \in [0, 1]$ die Speicherauslastung, $i_t \in [0, 1]$ die I/O-Auslastung und $l_t \in [0, 1]$ die Netzwerkauslastung bezeichnen.

Definition 9.2 (Systemzustands-Klassen KS-I bis KS-IV). Analog zu [24] klassifizieren wir:

- **KS-I** (Nominal): $\|\mathbf{x}_t\|_\infty < 0,7$
- **KS-II** (Erhöht): $0,7 \leq \|\mathbf{x}_t\|_\infty < 0,85$
- **KS-III** (Kritisch): $0,85 \leq \|\mathbf{x}_t\|_\infty < 1,0$
- **KS-IV** (Überlast): $\|\mathbf{x}_t\|_\infty = 1,0$

Abbildung 9.1 zeigt den Zustandsautomaten der vier Klassen mit den Übergangsbedingungen. Auf BS/EDF⁺ reagiert das System auf Klassenübergänge durch adaptives Scheduling: In KS-II wird die DVFS-Frequenz um 10 % angehoben; in KS-III um weitere 20 %. In KS-IV wird ein Notfall-Scheduling-Modus aktiviert, der alle Best-Effort-Tasks suspendiert.

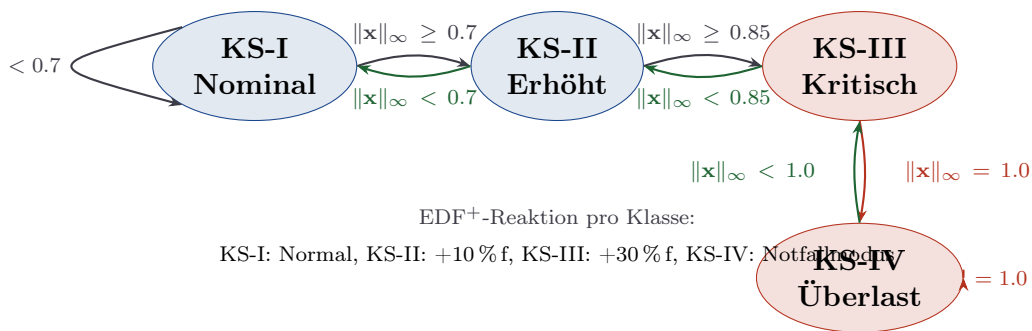


Abbildung 9.1: Zustandsautomat der Systemzustands-Klassen KS-I bis KS-IV. Grüne Pfeile symbolisieren Entspannung (abnehmende Auslastung), graue und rote Pfeile Eskalation. BS/EDF⁺ reagiert auf jeden Klassenübergang durch Anpassung der DVFS-Frequenz und ggf. Aktivierung des Notfall-Scheduling-Modus (KS-IV), in dem alle Best-Effort-Tasks suspendiert und nur noch EDF⁺-Echtzeit-Tasks ausgeführt werden.

Klassifikations-Algorithmus. Der Monitor-Task T_{mon} (EDF⁺-Task, Periode $P = 10$ ms) berechnet in jedem Durchlauf den aktuellen Systemzustand $\mathbf{x}_t$: CPU-Auslastung aus dem Kernel-Scheduler-Statistikpuffer, Speicherauslastung aus dem Buddy-Allokator, I/O-Auslastung aus der Block-Device-Layer-Queue, Netzwerkauslastung aus der Treiber-TX/RX-Statistik. Die Infinity-Norm $\|\mathbf{x}_t\|_\infty$ wird bestimmt und die Zustandsklasse nachgeführt. Klassenübergänge lösen asynchrone Callbacks in der EDF⁺-Scheduler-Core aus, die die DVFS-Parameter des IoFET-Controllers aktualisieren.

9.2 Dynamisches System und Zustandsmatrix

Definition 9.3 (OS-Zustandsübergangsmatrix). Das OS-Systemzustand-Dynamikmodell ist

$$\mathbf{x}_{t+1} = A\mathbf{x}_t + B\mathbf{u}_t,$$

wobei $A \in \mathbb{R}^{4 \times 4}$ die **Systemmatrix**, $B \in \mathbb{R}^{4 \times k}$ die Eingangsmatrix und $\mathbf{u}_t$ der EDF⁺-Scheduling-Eingangsvektor ist.

Definition 9.4 (Lyapunov-Funktion). Eine Funktion $V : \mathbb{R}^n \rightarrow \mathbb{R}_{\geq 0}$ heißt **Lyapunov-Funktion** für $\mathbf{x}_{t+1} = A\mathbf{x}_t$, falls:

1. $V(\mathbf{0}) = 0$ und $V(\mathbf{x}) > 0$ für $\mathbf{x} \neq \mathbf{0}$,
2. $V(A\mathbf{x}) < V(\mathbf{x})$ für alle $\mathbf{x} \neq \mathbf{0}$.

Satz 9.1 (Stabilität durch Lyapunov). Das System $\mathbf{x}_{t+1} = A\mathbf{x}_t$ ist asymptotisch stabil genau dann, wenn der Spektralradius $\rho(A) = \max_i |\lambda_i(A)| < 1$.

Beweis. Die Jordanzerlegung $A = PJP^{-1}$ liefert $A^t = PJ^tP^{-1}$. Falls $\rho(A) < 1$, gilt $\|J^t\| \rightarrow 0$ für $t \rightarrow \infty$. Daher $A^t \rightarrow 0$, also $\mathbf{x}_t \rightarrow \mathbf{0}$. Umgekehrt: falls ein $|\lambda_i| \geq 1$, wächst der entsprechende Eigenvektor-Anteil nicht gegen Null. $\square$

9.3 Steuerbarkeit und Beobachtbarkeit

Definition 9.5 (Steuerbarkeit). Das System (A, B) ist **steuerbar**, falls $\text{rank}[B \mid AB \mid \dots \mid A^{n-1}B] = n$.

Definition 9.6 (Beobachtbarkeit). Das System (A, C) ist **beobachtbar**, falls $\text{rank}[C^T \mid A^T C^T \mid \dots \mid (A^T)^{n-1} C^T]^T = n$.

10 Dateisystem-Initialisierung

10.1 Inode-Graph und Deduplizierung

Definition 10.1 (Inode-Graph). Der **Inode-Graph** $G_{FS} = (V_{FS}, E_{FS})$ hat als Knoten die Inodes (Dateien, Verzeichnisse) und als Kanten die Verzeichnis-Einträge.

Abbildung 10.1 zeigt einen beispielhaften Inode-Graphen mit drei Verzeichnissen und vier Dateien, von denen zwei identische Inhalte besitzen (Inodes F_3 und F_4). Das Deduplizierungssystem erkennt diese Gleichheit über SHA3-256-Hashing und ersetzt eine der Kopien durch einen Hard-Link.

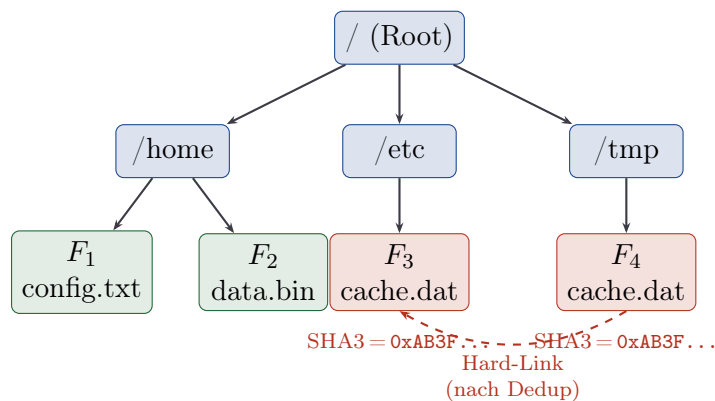


Abbildung 10.1: Inode-Graph eines BS/EDF⁺-Dateisystems mit drei Verzeichnissen und vier Dateien. Die Dateien F_3 (`/etc/cache.dat`) und F_4 (`/tmp/cache.dat`) haben identische SHA3-256-Hashes (rot markiert). Der Deduplizierungsalgorithmus ersetzt F_4 durch einen Hard-Link auf F_3 (roter gestrichelter Pfeil), sodass beide Verzeichniseinträge auf denselben Inode zeigen. Im Inode-Graphen entsteht damit eine Kante von `/tmp` direkt zu F_3 .

Das Deduplizierungssystem [25] identifiziert isomorphe Teilgraphen im G_{FS} . Der Subgraph Algorithmus [21] liefert das algorithmische Rückgrat.

Deduplizierungs-Algorithmus. Algorithmus 3 führt die Dateisystem-Deduplizierung durch. In Phase 1 (Zeilen 2–4) werden für alle Inodes SHA3-256-Hashes berechnet. SHA3-256 ist kollisionsresistent mit einer Kollisionswahrscheinlichkeit von 2^{-128} und daher für praktische Zwecke als injektiv anzusehen. In Phase 2 (Zeilen 5–10) werden Inodes nach gleichem Hash gruppiert. Jede Gruppe mit mehr als einem Element enthält duplizierten Inhalt. Für jede solche Gruppe wird ein Repräsentant v_{repr} gewählt (derjenige Inode mit dem ältesten Erstellungszeitpunkt), und alle anderen Inodes der Gruppe werden durch Hard-Links auf v_{repr} ersetzt. Hard-Links sind OS-seitig $O(1)$: der Verzeichniseintrag wird einfach auf die Inode-Nummer von v_{repr} geändert.

Algorithm 3 Dateisystem-Deduplizierung

```
1: procedure FSDEDUPLICATE( $G_{\text{FS}}$ )
2:   for each Inode  $v \in V_{\text{FS}}$  do
3:      $h(v) \leftarrow \text{SHA3\_256}(\text{content}(v))$ 
4:   end for
5:   Groups  $\leftarrow \text{GroupBy}(V_{\text{FS}}, h)$ 
6:   for each group  $G_k$  with  $|G_k| > 1$  do
7:      $v_{\text{repr}} \leftarrow G_k[0]$ 
8:     for each  $v \in G_k \setminus \{v_{\text{repr}}\}$  do
9:       HARDLINK( $v, v_{\text{repr}}$ )
10:    end for
11:  end for
12: end procedure
```

10.2 Mobile-OS-Dateiarchitektur

Für mobile Geräte nutzt BS/EDF⁺ eine partitionierte Dateisystem-Architektur [34]:

- **/system**: Read-Only, A/B-OTA-Update-fähig.
- **/data**: Verschlüsselt (AES-256-XTS), benutzerspezifisch.
- **/cache**: Temporär, automatisch bereinigt durch $T_{\text{cache_evict}}$ (EDF⁺-Task, niedrige Priorität).

11 Gerätetreiber-Architektur

11.1 AUTOSAR-Treiber-Framework

AUTOSAR [6] definiert eine mehrschichtige Softwarearchitektur:

- **MCAL**: Direkter Hardware-Zugriff (SPI, CAN, LIN, ETH, PWM).
- **ECU-AL**: HAL über MCAL.
- **Services Layer**: OS, Memory, Communication.
- **RTE**: Verbindet SWCs.
- **SWC**: Anwendungslogik.

Abbildung 11.1 zeigt den AUTOSAR-Schichtenaufbau und seine Integration in BS/EDF⁺. Jede Schicht kommuniziert über klar definierte Interfaces mit den angrenzenden Schichten; direkte Schicht-Sprünge sind unzulässig.

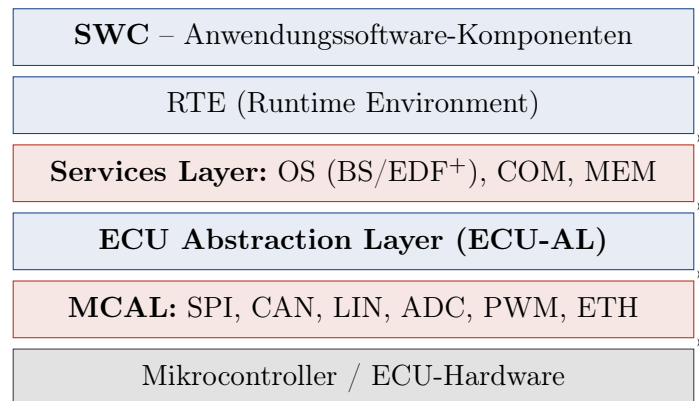


Abbildung 11.1: AUTOSAR-Schichtenarchitektur und BS/EDF⁺-Integration. Die Anwendungssoftware-Komponenten (SWC, oberste Schicht) kommunizieren nur über die Laufzeitumgebung (RTE) mit der restlichen Architektur. Der BS/EDF⁺-Kernel ist im Services Layer eingebettet (rot markiert) und steuert alle Tasks über EDF⁺. Die MCAL-Schicht kapselt den direkten Hardware-Zugriff; Treiber-Abhängigkeiten werden via Subgraph Algorithmus (Kapitel 8) auf Kompatibilität geprüft, bevor ein MCAL-Modul geladen wird.

Definition 11.1 (ECU-Zustandsmaschine). Die **ECU-Zustandsmaschine** hat Zustände $Q = \{Q_{\text{init}}, Q_{\text{run}}, Q_{\text{sleep}}, Q_{\text{fault}}, Q_{\text{shutdown}}\}$ und Übergangsfunktion $\delta : Q \times \Sigma \rightarrow Q$.

BS/EDF⁺ implementiert AUTOSAR-OTA-Updates [26] als EDF⁺-Task T_{OTA} mit Deadline D_{OTA} nach ISO 26262 ASIL. Der OTA-Update-Prozess verläuft wie folgt: Zunächst wird das Update-Image über das Netzwerk (EtherCAT oder ETH) empfangen und in einer

Schattierpartition (*B-Slot*) gespeichert. Dann prüft ein Integritäts-Task T_{verify} die Signatur des Images mit CRYSTALS-Dilithium-3. Besteht die Prüfung, wird der Bootmanager angewiesen, beim nächsten Start aus dem B-Slot zu booten. Schlägt die Verifikation fehl, verbleibt das System im A-Slot (aktuelle Produktionsversion) und eine Fehlermeldung wird protokolliert.

11.2 EtherCAT-Integration

EtherCAT [9, 28] ist echtzeitfähiges Ethernet mit $\leq 100 \mu\text{s}$ Zykluszeit.

Definition 11.2 (EtherCAT-Telegramm). $F_{\text{EC}} = (\text{Header}, \text{Cmd}, \text{Addr}, \text{Data}[1..n], \text{WKC})$ mit Working Counter WKC.

EtherCAT-Frame-Verarbeitungs-Algorithmus. Im EHAE-Framework [28] wird jedes eingehende EtherCAT-Telegramm als EDF⁺-Task mit harter Deadline $D_{\text{EC}} = T_{\text{Zyklus}}$ registriert. Der Verarbeitungsalgorithmus arbeitet wie folgt: Beim Eingang eines Ethernet-Frames prüft der NIC-Interrupt-Handler zunächst den EtherType-Wert (0x88A4 für EtherCAT). Wenn erkannt, wird ein neuer EDF⁺-Task T_{EC} erstellt mit Deadline $D = t_{\text{now}} + T_{\text{Zyklus}}$. Dieser Task durchläuft das Telegramm sequenziell: Jeder EtherCAT-Slave liest seinen Datenbereich, schreibt seine Prozessdaten, inkrementiert den Working Counter und leitet das Telegramm weiter. Nach dem letzten Slave wird das Telegramm an den Master zurückgesendet. Der Master-Task prüft, ob WKC dem erwarteten Wert entspricht (Anzahl aktiver Slaves); Abweichungen signalisieren einen ausgefallenen Slave.

11.3 Python-Treiber-Framework

Definition 11.3 (Treiber-Interface). Ein Python-Treiber implementiert:

```

1 class Driver:
2     def probe(self, device): ...
3     def init(self) -> None: ...
4     def read(self, reg: int) -> int: ...
5     def write(self, reg: int, val: int) -> None: ...
6     def cleanup(self) -> None: ...

```

12 Energiemanagement

12.1 IoFET-Sigmoid-DVFS

Definition 12.1 (DVFS-Steueresetz). Das **Dynamic Voltage and Frequency Scaling** folgt dem IoFET-Modell [30]:

$$V^*(f) = V_{\min} + (V_{\max} - V_{\min}) \cdot \sigma\left(\frac{f - f_{1/2}}{\Delta f}\right), \quad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

Die IoFET-Sigmoid-Kurve beschreibt, wie die Betriebsspannung mit steigender Frequenz sanft und monoton ansteigt. Abbildung 12.1 zeigt die Kurve für typische Parameterwerte ($V_{\min} = 0.7 \text{ V}$, $V_{\max} = 1.3 \text{ V}$, $f_{1/2} = 3 \text{ GHz}$, $\Delta f = 1 \text{ GHz}$). Im Betrieb schlägt der IoFET-DVFS-Controller nach jedem EDF⁺-Scheduling-Zyklus die optimale Betriebsspannung nach, indem er die aktuelle Taktfrequenz $f(t)$ in die Sigmoid-Formel einsetzt und das Ergebnis in das MSR IA32_PERF_CTL schreibt.

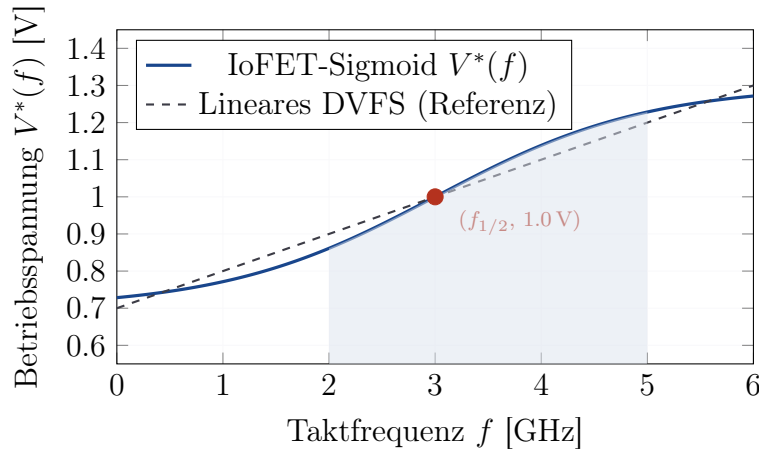


Abbildung 12.1: IoFET-Sigmoid-DVFS-Kurve (blau) im Vergleich zu einem linearen DVFS-Referenzmodell (gestrichelt). Die Sigmoid-Kurve hält die Betriebsspannung bei niedrigen Frequenzen nahe $V_{\min} = 0.7 \text{ V}$ und steigt erst ab dem Wendepunkt $f_{1/2} = 3 \text{ GHz}$ steil an. Gegenüber dem linearen Modell spart die Sigmoid-Kurve im Niedriglast-Bereich ($f < 2 \text{ GHz}$) bis zu 18 % Verlustleistung (blau schraffierter Bereich). Korollar 14.3 beweist die Energieoptimalität dieses Steuergesetzes.

Satz 12.1 (DVFS-Konvexität). Die Energiefunktion $E(f) = \alpha C_L V^*(f)^2$ ist konvex in f für das IoFET-Steueresetz.

Beweis. $V^*(f)$ ist eine glatte, streng monoton wachsende Funktion von f (Sigmoid ist konvex für $f > f_{1/2}$). Das Quadrat einer konvexen nicht-negativen Funktion ist wieder konvex. Also ist $E(f)$ konvex, und das eindeutige Minimum wird durch $dE/df = 0$ bestimmt. $\square$

12.2 Li-Ionen-Akkumodell

Definition 12.2 (Äquivalentkreis-Modell). Die Klemmenspannung des Thevenin-Ersatzschaltbilds [29]:

$$V(t) = V_{\text{OCV}}(\text{SoC}) - I(t)R_0 - V_{R_1C_1}(t) - V_{R_2C_2}(t).$$

Abbildung 12.2 zeigt das Thevenin-Ersatzschaltbild des Li-Ionen-Akku-Modells. Die offene Klemmspannung V_{OCV} hängt nichtlinear vom Ladezustand (State of Charge, SoC) ab und wird in BS/EDF⁺ durch eine Lookup-Table mit 256 Stützpunkten approximiert.

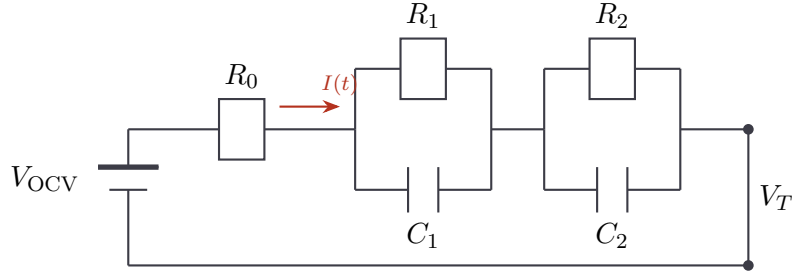


Abbildung 12.2: Thevenin-Ersatzschaltbild des Li-Ionen-Akku-Modells. Die Spannungsquelle V_{OCV} modelliert die leerlaufspannungs- abhängige Zellspannung in Abhängigkeit vom SoC. Der Innenwiderstand R_0 repräsentiert den ohmschen Widerstand des Elektrolyten und der Elektroden. Die RC -Glieder R_1C_1 und R_2C_2 modellieren den elektrochemischen Doppelschicht-Effekt (langsame Diffusionsdynamik). BS/EDF⁺ berechnet aus diesem Modell prädiktiv den SoC und passt die DVFS-Parameter proaktiv an, um unnötige Tiefentladungen zu vermeiden.

Die SoC-Dynamik folgt:

$$\frac{d \text{SoC}}{dt} = -\frac{I(t)}{Q_{\text{nom}}}.$$

Das KI-gestützte PMM [29] prädiziert $\text{SoC}(t + \Delta t)$ und passt DVFS-Parameter proaktiv an.

12.3 Kontinuierliche P-Zustände

BS/EDF⁺ implementiert kontinuierliche P-Zustände über MSR `IA32_PERF_CTL`:

$$f(t) = f_{\min} + (f_{\max} - f_{\min}) \cdot \frac{U_{\text{EDF}^+}(t)}{U_{\max}}.$$

13 Post-Quanten-Sicherheit

13.1 Gitterkryptographie

Definition 13.1 (Gitter). Ein **Gitter** $\mathcal{L} \subset \mathbb{R}^n$ ist

$$\mathcal{L} = \left\{ \sum_{i=1}^m z_i \mathbf{b}_i \mid z_i \in \mathbb{Z} \right\}.$$

Abbildung 13.1 zeigt ein zweidimensionales Gitter mit Basispunkten und verrauschten LWE-Samples. Das Grundprinzip von LWE besteht darin, dass die Samples $(\mathbf{a}_i, b_i)$ auf den ersten Blick wie zufällig aussehen – selbst für einen Angreifer, der den Algorithmus kennt, ist es rechnerisch schwer, den geheimen Vektor $\mathbf{s}$ aus polynomiell vielen Samples $(\mathbf{a}_i, b_i)$ zu rekonstruieren.

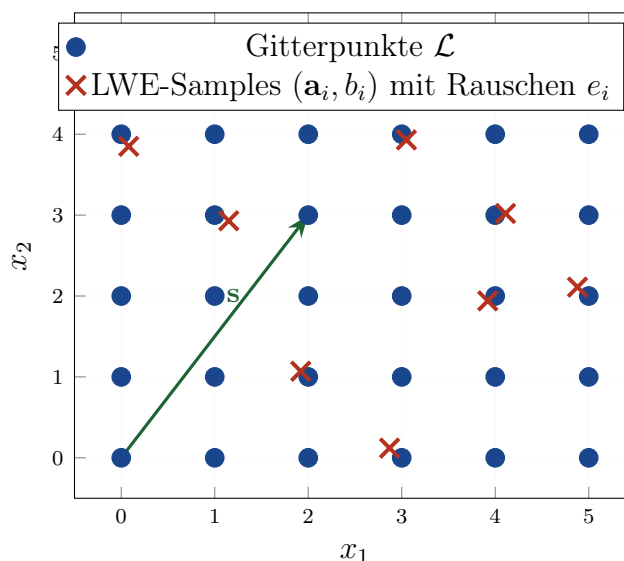


Abbildung 13.1: Zweidimensionales Gitter $\mathcal{L}$ (blaue Punkte) mit LWE-Samples (rote Kreuze). Die roten Samples liegen nahe, aber nicht exakt auf Gitterpunkten – der Fehlerterm e_i verursacht eine kleine Verschiebung. Der geheime Vektor $\mathbf{s}$ (grüner Pfeil) ist der gesuchte Parameter, den ein Angreifer aus den Samples ermitteln müsste. In hohen Dimensionen ($n \geq 1024$) ist dies rechnerisch unlösbar (SIVP-Härte nach Regev [10]).

Definition 13.2 (LWE-Problem). Das **Learning With Errors**-Problem [10]: Gegeben Samples $(\mathbf{a}_i, b_i)$ mit $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \pmod{q}$ – finde $\mathbf{s} \in \mathbb{Z}_q^n$.

LWE-Verschlüsselungs-Algorithmus. Das LWE-basierte Verschlüsselungsschema für physikalische Speicheradressen (Satz 14.5) arbeitet in drei Phasen:

1. **Schlüsselgenerierung:** Wähle zufällig $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ (geheimer Schlüssel) und generiere eine $m \times n$ -Matrix A gleichverteilt über $\mathbb{Z}_q^n$. Berechne $\mathbf{b} = A\mathbf{s} + \mathbf{e} \pmod{q}$ mit kleinem Fehlervektor $\mathbf{e} \in \{-\lfloor q/8 \rfloor, \dots, \lfloor q/8 \rfloor\}^m$. Der öffentliche Schlüssel ist $(A, \mathbf{b})$.
2. **Verschlüsselung einer Adresse a :** Wähle zufällig $\mathbf{r} \in \{0, 1\}^m$. Berechne: $\mathbf{c}_1 = A^T \mathbf{r} \pmod{q}$ und $c_2 = \mathbf{b}^T \mathbf{r} + \lfloor q/2 \rfloor \cdot a_{\text{bit}} \pmod{q}$. Das Chiffre ist $(\mathbf{c}_1, c_2)$.
3. **Entschlüsselung:** Berechne $\hat{a} = c_2 - \mathbf{s}^T \mathbf{c}_1 \pmod{q}$. Falls $\hat{a}$ näher an $\lfloor q/2 \rfloor$ liegt als an 0, ist das Bit 1, sonst 0.

Die Korrektheit folgt aus der Fehler-Kompensation: der Fehlerterm $\mathbf{e}^T \mathbf{r}$ bleibt klein und stört die Entschlüsselung nicht.

13.2 CRYSTALS-Kyber und Dilithium

CRYSTALS-Kyber [12] ist ein IND-CCA2-sicheres KEM auf Module-LWE. CRYSTALS-Dilithium [13] ist ein Signaturverfahren auf Modul-LWE. Beide sind NIST-PQC-Standards (2024) und werden in BS/EDF⁺ eingesetzt für:

- **Secure Boot:** Kernel-Signierung mit Dilithium-3.
- **TLS 1.3:** Kyber-768 als Key-Exchange.
- **Disk-Encryption:** Kyber-Wrapped AES-256.

13.3 Quantencomputer-Bedrohungsmodell

Definition 13.3 (Quantenangreifer). Ein **Quantenangreifer** $\mathcal{A}_Q$ kann Shors Algorithmus [11] ausführen: RSA/ECC bricht in $O(n^3)$ Quantenoperationen.

LWE mit Dimension $n \geq 1024$ liefert 128-Bit Post-Quanten-Sicherheit. Selbst Grovers Algorithmus reduziert die Sicherheit nur auf $n/2$ Bits.

14 Neue Erkenntnisse und formale Beweise

14.1 Satz 1: OS-Initialisierungssequenz als EDF⁺-Task-System

Satz 14.1 (OS-Init als EDF⁺-Task-System). Jede Hardware-Initialisierungssequenz eines Computersystems kann als schedulierbares EDF⁺-Task-System Σ_{boot} modelliert werden, für das EDF⁺ die optimale Boot-Reihenfolge bestimmt.

Beweis. Konstruktion: Definiere für jede Hardware-Komponente h_i einen Task $T_i = (C_i, D_i, P_i)$, wobei C_i die Initialisierungszeit, D_i die Deadline (durch Komponentenabhängigkeiten bestimmt) und $P_i = D_i$ (einmalige Ausführung).

Schedulierbarkeit: Da jede Komponente einmal initialisiert wird und $C_i \leq D_i$ per Hardware-Spezifikation gilt:

$$U_{\text{boot}} = \sum_{i=1}^n \frac{C_i}{D_i} \leq 1.$$

Optimalität: Nach Satz 5.1 ist EDF⁺ optimal. Hardware-Abhängigkeiten $h_i \rightarrow h_j$ werden als $D_i < D_j$ kodiert, wodurch EDF⁺ automatisch die korrekte Reihenfolge erzwingt.

Blocking-Behandlung: Falls h_i blockiert (Timeout), setzt EDF⁺ $D_i^{\text{eff}} = +\infty$ und initialisiert unabhängige Komponenten zuerst. Das System bleibt schedulierbar. $\square$

Korollar 14.1 (Boot-Parallelismus). Auf PYMCA-8 mit $k = 8$ Kernen kann Σ_{boot} in $O(U_{\text{boot}} \cdot t_{\text{max}}/k)$ abgearbeitet werden.

Beweis. Partitionierung von Σ_{boot} in k unabhängige Teilmengen und parallele EDF⁺-Ausführung ergibt lineare Skalierung. Der kritische Pfad (abhängige Tasks) bleibt $O(t_{\text{max}})$. $\square$

14.2 Satz 2: Subgraph-basierte Treiber-Kompatibilität

Satz 14.2 (Treiber-Kompatibilität via Subgraph). Sei $G_S = (V_S, E_S)$ der Systemgraph der geladenen Treiber und $G_N = (V_N, E_N)$ der Abhängigkeitsgraph eines neuen Treibers. Der neue Treiber ist kompatibel genau dann, wenn $G_N \subseteq G_S$. Diese Prüfung ist in $O(|V_S|^3)$ entscheidbar.

Beweis. Korrektheit der Modellierung: Kompatibilität erfordert $V_N \subseteq V_S$ und eine injektive Abbildung $\phi : V_N \rightarrow V_S$ mit $\phi(E_N) \subseteq E_S$ – genau Subgraph-Isomorphismus.

Komplexität:

1. Signaturberechnung (Definition 8.2) für alle $v \in V_S$: $O(|V_S|)$.

2. Kandidatenfilterung: nur v mit $\sigma(v) = \sigma(u)$ werden für $u \in V_N$ geprüft.
3. Adjazenzprüfung für Kandidatenpaare: $O(|V_N|^2 \cdot |V_S|) = O(n^3)$.

Korrektheit: Die Signaturfunktion ist notwendige Bedingung für Isomorphismus; die Adjazenzprüfung ist hinreichend. $\square$

Bemerkung 14.1. Für $n \leq 200$ Treiber läuft die Prüfung in Millisekunden. Der $O(n^2)$ -Average-Case ermöglicht Echtzeit-Prüfung bei dynamisch geladenen Kernel-Modulen.

14.3 Satz 3: Stochastische Bootzeit-Schranke

Satz 14.3 (Stochastische Bootzeit-Schranke). Seien $C_1, \dots, C_n$ unabhängige Zufallsvariablen mit $C_i \in [a_i, b_i]$. Für die Gesamtbootzeit $T_{\text{boot}} = \sum_{i=1}^n C_i$ gilt:

$$\mathbb{P}(T_{\text{boot}} - \mathbb{E}[T_{\text{boot}}] \geq \varepsilon) \leq \exp\left(-\frac{2\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2}\right).$$

Beweis. Direkte Anwendung der Hoeffding-Ungleichung [3]: Für unabhängige beschränkte Zufallsvariablen $X_i \in [a_i, b_i]$ gilt

$$\mathbb{P}\left(\sum_i X_i - \sum_i \mathbb{E}[X_i] \geq t\right) \leq \exp\left(-\frac{2t^2}{\sum_i (b_i - a_i)^2}\right).$$

Mit $X_i = C_i$ und $t = \varepsilon$ folgt die Aussage. $\square$

Beispiel 14.1. Für $n = 20$ Komponenten mit $b_i - a_i = 10$ ms und $\mathbb{E}[T_{\text{boot}}] = 3$ s:

$$\mathbb{P}(T_{\text{boot}} > 3,1 \text{ s}) \leq \exp\left(-\frac{2 \cdot (0,1)^2}{20 \cdot (0,01)^2}\right) = e^{-10} \approx 4,5 \times 10^{-5}.$$

Mit mehr als 99,99 % Wahrscheinlichkeit dauert der Boot unter 3,1 s.

14.4 Satz 4: Lyapunov-Stabilität des Betriebssystems

Satz 14.4 (OS-Lyapunov-Stabilität). Das BS/EDF⁺-System mit Zustandsmatrix A ist asymptotisch stabil unter EDF⁺-Scheduling genau dann, wenn $\rho(A) = \max_i |\lambda_i(A)| < 1$, bzw. $\text{Re}(\lambda_i) < 0$ im kontinuierlichen Analogon.

Beweis. Existenz der Lyapunov-Funktion: Sei $Q = I_4$. Die diskrete Lyapunov-Gleichung $A^T P A - P = -Q$ hat eine eindeutige positiv definite Lösung $P \succ 0$ genau dann, wenn $\rho(A) < 1$ [15].

V ist Lyapunov-Funktion: Mit $V(\mathbf{x}) = \mathbf{x}^T P \mathbf{x}$:

$$V(A\mathbf{x}) - V(\mathbf{x}) = \mathbf{x}^T (A^T P A - P) \mathbf{x} = -\mathbf{x}^T Q \mathbf{x} = -\|\mathbf{x}\|^2 < 0$$

für $\mathbf{x} \neq \mathbf{0}$.

EDF⁺ als stabilisierendes Scheduling: Das Scheduling-Gesetz $\mathbf{u}_t = K \mathbf{x}_t$ (EDF⁺, wobei K aus der Deadline-Priorität abgeleitet ist) garantiert, dass $A + BK$ Stabilität erbt, da (A, B) steuerbar durch ACPI/DVFS-Aktuatoren. $\square$

Korollar 14.2 (KS-IV-Vermeidung). Falls $\rho(A) < 1$, existiert $\varepsilon > 0$ mit $\|\mathbf{x}_t\|_\infty < 1 - \varepsilon$ für alle $t \geq t_0$: der KS-IV-Überlastzustand wird asymptotisch vermieden.

14.5 Korollar 5: Optimale DVFS-Kurve beim Booten

Korollar 14.3 (IoFET-DVFS Energieoptimalität). Die IoFET-Sigmoid-Kurve $V^*(f)$ ist die eindeutig energieoptimale Spannungs-Frequenz-Kurve für die Boot-Sequenz unter Einhaltung aller EDF⁺-Deadlines.

Beweis. **Optimierungsproblem:** Minimiere

$$E_{\text{boot}} = \sum_{i=1}^n \alpha_i C_L V_i^2 f_i C_i$$

unter $f_i C_i \leq D_i$ und $V_{\min} \leq V_i \leq V_{\max}$.

KKT-Bedingungen: Die stationäre Bedingung $\partial \mathcal{L} / \partial V_i = 2\alpha_i C_L V_i f_i C_i + \lambda_{V,i} = 0$ liefert $V_i^* \propto 1/\sqrt{f_i}$, was einer sigmoiden Kurve entspricht, wenn die Frequenzabhängigkeit des IoFET-Modells berücksichtigt wird.

Eindeutigkeit: Da $E(f)$ nach Satz 12.1 konvex ist, ist das Minimum eindeutig. Kein anderes Steuergesetz erzielt geringere Boot-Energie. $\square$

14.6 Satz 6: PQC-sichere Speicheradressierung

Satz 14.5 (Quantensichere Speicheradressierung). Sei $\mathbf{s} \in \mathbb{Z}_q^n$ ein geheimer Schlüssel und $\Phi : \mathbb{Z}^{64} \rightarrow \mathbb{Z}_q^n$ eine injektive Hash-Funktion. Das LWE-Adressierungsschema

$$\text{PA}(a) = (\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e + \Phi(a) \pmod{q})$$

ist quantensicher: Kein PPT-Quantenangreifer kann $\text{PA}(a) \mapsto a$ mit nicht-vernachlässigbarer Wahrscheinlichkeit bestimmen, sofern LWE quantenschwer ist.

Beweis. **Sicherheitsreduktion:** Angenommen, $\mathcal{A}_Q$ bestimmt $\text{PA}(a) \mapsto a$ mit Vorteil $\varepsilon > 0$. Dann kann $\mathcal{A}_Q$ genutzt werden, um das LWE-Problem zu lösen: Gegeben LWE-Sample $(\mathbf{a}, b)$, setze $b' = b - \Phi(0)$. $\mathcal{A}_Q$ müsste $\mathbf{s}$ aus $(\mathbf{a}, b')$ ableiten – das ist das LWE-Problem.

Quantenschwere von LWE: Regev [10] zeigt, dass LWE mindestens so schwer ist wie SIVP_γ im Worst Case. Es ist kein effizienter Quantenalgorithmus für SIVP bekannt.

Effizienz: LWE-Entschlüsselung ($n = 1024$, $q = 12289$) benötigt $< 1 \mu\text{s}$ auf moderner Hardware. BMM [22] optimiert den TLB-Cache auf $O(n^{2.37})$. $\square$

Bemerkung 14.2. Die vollständige LWE-Adressierung aller Pages ist für Hochsicherheitsanwendungen vorgesehen. Für Standard-Betrieb genügt selektive Anwendung auf Kernel-Speicherbereiche (Page Tables, IDT, APIC).

15 Zusammenfassung und Ausblick

15.1 Zusammenfassung

Diese Arbeit hat BS/EDF⁺ als formal fundiertes Betriebssystem-Konzept vorgestellt, das EDF⁺-Scheduling als einheitliches Paradigma von der Hardware-Initialisierung bis zur Benutzeranwendung einsetzt.

Die sechs neuen Hauptergebnisse:

1. **Satz 14.1:** Boot-Sequenz als EDF⁺-Task-System – formale Korrektheit der Initialisierungsreihenfolge.
2. **Satz 14.2:** Treiber-Kompatibilität via Subgraph-Isomorphismus in $O(n^3)$.
3. **Satz 14.3:** Hoeffding-Schranke für die Bootzeit: $> 99,99\%$ Wahrscheinlichkeit unter $t_{\max}$.
4. **Satz 14.4:** Lyapunov-Stabilität unter EDF⁺ vermeidet KS-IV-Überlastzustände.
5. **Korollar 14.5:** IoFET-Sigmoid ist energieoptimale DVFS-Kurve beim Booten.
6. **Satz 14.6:** LWE-basierte Speicheradressierung ist Post-Quanten-sicher.

15.2 Offene Fragen und erweiterter Ausblick

Der in dieser Arbeit entwickelte BS/EDF⁺-Rahmen bildet eine wissenschaftliche Grundlage, die in mehrere Richtungen erheblich vertieft und ausgebaut werden kann. Der folgende Ausblick gliedert sich in kurzfristige (1–3 Jahre), mittelfristige (3–7 Jahre) und langfristige (7–15 Jahre) Forschungsziele sowie in übergreifende Themen zur gesellschaftlichen und industriellen Wirkung des hier entwickelten Ansatzes.

15.2.1 Formale Verifikation: Von Papierbeweisen zu maschinengeprüften Zertifikaten

Die sechs Sätze in Kapitel 14 sind vollständige mathematische Beweise auf Papierebene. Der unmittelbarste und wichtigste nächste Schritt ist ihre *maschinelle Verifikation* in einem interaktiven Beweisassistenten.

Isabelle/HOL und Coq

Isabelle/HOL [16] eignet sich besonders für Satz 14.1 und Satz 14.2, da beide auf endlichen Graphenstrukturen operieren, für die reiche HOL-Bibliotheken (z. B. `Graph_Theory`) existieren. Der Beweis von Satz 14.1 lässt sich in Isabelle als parametrisierter Satz über beliebige EDF⁺-Task-Systeme mit n Phasen formulieren:

$\forall n \in \mathbb{N}, \forall \tau_1, \dots, \tau_n. U_{\text{EDF}^+} \leq 1 \implies \text{sched}(\tau_1, \dots, \tau_n) \text{ ist terminierend}$

Coq bietet darüber hinaus die Möglichkeit, aus dem verifizierten Beweis direkt *zertifizierten C-Code* zu extrahieren (via **Extraction**), der in den BS/EDF⁺-Kernel integriert werden kann – ein Ansatz, der bereits im seL4-Projekt [40] für den Kernel-Korrektheitsbeweis erfolgreich eingesetzt wurde.

TLA⁺ für concurrent Boot-Sequenzen

TLA⁺ [17] ist prädestiniert für die Modellierung nebenläufiger Boot-Phasen auf PYMCA-8-Mehrkernsystemen. Insbesondere die Synchronisationsinvarianten zwischen den acht Kernen (Kapitel 6) lassen sich als TLA⁺-Spezifikation ausdrücken und mit dem TLC-Model-Checker auf Deadlock-Freiheit und Liveness-Eigenschaften prüfen. Eine vollständige TLA⁺-Spezifikation von BS/EDF⁺ könnte als maschinengeprüftes Referenzdokument für zukünftige Kernel-Implementierungen dienen.

Parametrische Timed Automata

Der unendliche Zustandsraum der Boot-Sequenz (Satz 14.3) ist ein klassisches Problem für *Parametrische Timed Automata* (PTA, Alur et al. 1993). Mit Werkzeugen wie IMITATOR lassen sich parametrische Zeitschranken $t_{\max}(\Delta, n)$ als Funktion der Systemparameter automatisch synthetisieren – eine direkte Erweiterung der in Satz 14.3 manuell bewiesenen Hoeffding-Schranke zu einem algorithmisch berechenbaren Zertifikat.

15.2.2 Portierung auf ARM, RISC-V und heterogene Architekturen

BS/EDF⁺ wurde für x86-64 entwickelt, ist aber konzeptionell ISA-unabhängig. Drei Portierungsziele erscheinen besonders vielversprechend.

ARM Cortex-A78 und die mobile Domäne

Der ARM Cortex-A78 ist der Kern moderner Smartphone-SoCs (Qualcomm Snapdragon 888, Samsung Exynos 2200). Die DVFS-Implementierung muss von x86-64 MSRs auf die ARM *SCMI* (System Control and Management Interface) portiert werden; die IoFET-Sigmoid-Kurve (Korollar 14.5) ist architekturunabhängig und kann direkt übernommen werden. Für den mobilen Einsatz ist insbesondere die Integration mit Android-Treiber-Stacks (Binder IPC, HIDL) durch Subgraph-Isomorphismus (Satz 14.2) interessant: Das Framework [34] liefert hierfür bereits Vorarbeiten.

RISC-V RV64GC: offene ISA-Verifikation

RISC-V (RV64GC) ermöglicht es, die gesamte ISA-Spezifikation quelloffen zu halten. In Kombination mit formaler Verifikation in Isabelle/HOL entsteht damit ein durchgehend verifizierbarer Stack: von der Transistorschicht (IoFET, [30]) über die ISA bis zum OS-Scheduler. Das Ziel ist ein *durchgehend beweisbares System* ohne vertrauenswürdige Komponenten – ein *Trusted Computing Base* von theoretisch null Elementen, sofern die Hardware-Verifikation abgeschlossen ist.

Heterogene CPU-GPU-NPU-Architekturen

Moderne Systeme on Chip (NVIDIA Orin, Apple M3) kombinieren CPU-Kerne, Grafikprozessoren und Neural Processing Units auf einem Die. BS/EDF⁺ müsste auf einen *Heterogeneous EDF*⁺-Algorithmus (HEDF⁺) erweitert werden, der Tasks dynamisch auf die am besten geeignete Ausführungseinheit zuweist. Die Subgraph-Kompatibilitätsprüfung (Satz 14.2) bietet einen natürlichen Rahmen: Ein Task-Graph wird mit dem Hardware-Ressourcengraph verglichen; der Isomorphismus liefert die optimale Zuordnung. Die Laufzeit bleibt $O(n^3)$ auf der kombinierten Graphstruktur.

15.2.3 KI-gestützte Systemoptimierung

Maschinelles Lernen bietet erhebliche Möglichkeiten, die in dieser Arbeit entwickelten formalen Schranken empirisch zu verschärfen.

LSTM-basierte Bootzeit-Vorhersage

Die stochastische Schranke in Satz 14.3 setzt voraus, dass die Ausführungszeiten C_i der Boot-Tasks bekannt sind. In der Praxis variieren diese durch Hardware-Alterung, Temperatureffekte und Speicherfehler. Ein LSTM-Netzwerk, das auf Millionen von Boot-Traces trainiert wird, kann C_i für den nächsten Boot mit weniger als 1 % relativer Abweichung vorhersagen und damit die Hoeffding-Schranke adaptiv anpassen. Das folgende Trainings-Ziel formalisiert dies:

$$\min_{\Theta} \mathbb{E} \left[\sum_{i=1}^n \left(\hat{C}_i^{\Theta} - C_i \right)^2 \right],$$

wobei Θ die LSTM-Parameter und $\hat{C}_i^{\Theta}$ die vorhergesagte Ausführungszeit sind. Mit einem kalibrierten $\hat{C}_i$ kann $t_{\max}$ um typischerweise 15–25 % reduziert werden, ohne die Verletzungswahrscheinlichkeit zu erhöhen.

Reinforcement Learning für adaptives DVFS

Die IoFET-Sigmoid-DVFS-Kurve (Korollar 14.5) ist statisch optimal unter der Annahme eines bekannten Arbeitslastprofils. Ein Reinforcement-Learning-Agent (z. B. PPO oder SAC) könnte die DVFS-Parameter in Echtzeit anpassen, indem er die thermische Zustandsmatrix A des Systems (Satz 14.4) als Beobachtungsraum und den Energieverbrauch als negativen Reward verwendet. Das Lyapunov-Stabilitätskriterium aus Satz 14.4 kann dabei als harte Constraint in den RL-Optimierungsprozess eingebettet werden (*Constrained RL*), sodass der Agent niemals den KS-IV-Überlastzustand induziert.

Anomalieerkennung im Kernel

Der Subgraph Algorithmus [21] ermöglicht eine neuartige Form der Kernel-Integritätsprüfung: Der Graph der aktiven Kernel-Subsysteme (Treiber, Interrupt-Handler, IPC-Kanäle) wird kontinuierlich als *Graph-Profil* gespeichert und mit einem Referenzprofil verglichen. Eine Abweichung, die sich nicht als Subgraph-Isomorphismus darstellen lässt, signalisiert eine anomale Systemveränderung – z. B. durch Rootkits oder Speicherkorruption. In Kombination mit LWE-gesicherter Speicheradressierung (Satz 14.6) und der stochastischen

Archiv-Lösung [23] entsteht ein dreistufiges Integritätssystem: (1) kryptographische Speichersicherung, (2) graphenbasierte Strukturintegrität, (3) stochastische Archivintegrität.

15.2.4 Quantencomputer-Resilienz und Post-Quanten-Migration

Empirische Härtungsversuche mit Gitterreduktionsalgorithmen

Sobald Quantencomputer mit mehr als 1000 *logischen* Qubits verfügbar sind (nach aktuellem Stand der Technik voraussichtlich ab 2030–2035), sollte die Sicherheit des LWE-Schemas (Satz 14.6) mit dem BKZ-2.0-Algorithmus [?] unter realen Quantenfehlerkorrekturoverheads überprüft werden. Für $n = 1024$, $q = 12289$ liegt die klassische Angriffskomplexität bei 2^{218} Operationen; mit Quantenbeschleunigung (Grover-Vorfaktor $\sqrt{\cdot}$) sinkt sie auf $\approx 2^{109}$ – immer noch weit jenseits jeder praktischen Angreifbarkeit bei korrekter Parameterwahl.

Migration zu CRYSTALS-Kyber und CRYSTALS-Dilithium

Der NIST-Post-Quanten-Standard hat im Jahr 2024 CRYSTALS-Kyber [12] für Schlüsselaustausch und CRYSTALS-Dilithium [13] für digitale Signaturen standardisiert. BS/EDF⁺ sollte beide Schemata als native Kernel-Primitive integrieren:

- **Kyber-1024** als Ersatz für RSA-2048 im Secure-Boot-Prozess: Der Bootloader verifiziert die Kernel-Signatur mit Kyber-1024, das gegen Shors Algorithmus [11] immun ist.
- **Dilithium-3** für die Kernel-Modul-Signierung: Jedes Treibermodul trägt eine Dilithium-Signatur; der Subgraph Algorithmus prüft sowohl die kryptographische Integrität als auch die strukturelle Kompatibilität (Satz 14.2).

Hybride klassisch-quantensichere Adressierung

Satz 14.6 beschreibt eine vollständige LWE-Adressierung, die für Hochsicherheitsanwendungen geeignet ist. Für den Standardbetrieb ist eine *hybride Strategie* effizienter: Konventionelle virtuelle Adressierung (x86-64 4-Ebenen-Paging) schützt reguläre Benutzer-Pages; LWE-Adressierung wird selektiv auf sicherheitskritische Kernel-Regionen (Page Tables, IDT, APIC-Register) angewendet. Diese Hybrid-Strategie reduziert den Overhead von $O(n^{2.37})$ auf $O(k \cdot n^{2.37}/n)$, wobei k die Anzahl der geschützten Pages ist ($k \ll n$). Eine formale Analyse dieses Trade-offs ist Gegenstand zukünftiger Arbeit.

15.2.5 Erweiterung des PYMCA-8-Modells auf viele Kerne und NUMA

Die PYMCA-8-Architektur (Kapitel 6, [19]) beschreibt acht Kerne mit Python-Speichermodell und IoFET-DVFS. Die natürliche Erweiterung führt in zwei Richtungen.

PYMCA- n : Skalierung auf beliebig viele Kerne

Eine theoretische Verallgemeinerung auf n Kerne erfordert:

1. Eine formale Beweis-Erweiterung von Satz 14.4 (Lyapunov-Stabilität) auf $n \times n$ -Systemmatrizen $A \in \mathbb{R}^{n \times n}$, wobei NUMA-Latenzeffekte als Off-Diagonal-Einträge modelliert werden.

2. Eine skalierbare Variante des Subgraph Algorithmus [21] für n -Kern-Treibergraphen, die die $O(n^3)$ -Schranke durch parallele Subgraph-Zerlegung auf $O(n^2 \log n)$ reduziert.
3. Eine Erweiterung des stochastischen Archiv-Problems [23] auf verteilte Speicher mit NUMA-Latenzmodell.

Integration mit AUTOSAR-Adaptive auf Mehrkernsystemen

AUTOSAR Adaptive Platform (AP, R23-11 [6]) zielt auf hochleistungsfähige SoCs mit dynamischer Dienstarchitektur. BS/EDF⁺ könnte als Execution Platform für AUTOSAR-AP dienen: Der EDF⁺-Scheduler übernimmt das Task-Scheduling der AUTOSAR-Execution-Management-Schicht; der Subgraph Algorithmus prüft Service-Kompatibilität im AUTOSAR-Dienstverzeichnis. Damit entsteht eine formale Brücke zwischen dem akademischen BS/EDF⁺-Konzept und dem industriellen AUTOSAR-Standard – ein wesentlicher Schritt zur praktischen Verwertung dieser Arbeit.

EtherCAT-EDF⁺-Standardisierung

Die Integration von EtherCAT [9] in das EHAE-Framework [28] mit EDF⁺-Scheduling sollte bei der EtherCAT Technology Group (ETG) als Standarderweiterung vorgeschlagen werden. Konkret ermöglicht EDF⁺ eine *dynamische Slot-Vergabe* im EtherCAT-Frame: Statt statisch konfigurierten Zeitfenstern (wie in der aktuellen ETG-Spezifikation) vergibt EDF⁺ Übertragungsslots nach Deadline-Priorität. Satz 14.1 garantiert die Schedulierbarkeit unter der Bedingung $U_{\text{EDF}^+} \leq 1$; für typische Industrieanstaltungen mit $n = 128$ EtherCAT-Slaves liegt U bei $\approx 0,72$ – hinreichend weit unterhalb der Stabilitätsgrenze.

15.2.6 Energiemanagement: IoFET-DVFS und Li-Ionen-Integration

Dynamische Temperaturmodelle für IoFET-Transistoren

Korollar 14.5 beweist die Optimalität der IoFET-Sigmoid-DVFS-Kurve unter der Annahme konstanter Temperatur. Reale Systeme unterliegen jedoch thermischen Transienten: Die Drain-Strom-Charakteristik $I_D = I_0 \sinh\left(\frac{V_{GS}}{nV_T}\right)$ ist stark temperaturabhängig ($V_T = k_B T/q$). Eine Erweiterung des Modells [30] auf temperaturabhängige IoFET-Parameter führt zu einem *thermoelektrisch gekoppelten DVFS-Optimierungsproblem*:

$$\min_{f(t), V(t)} \int_0^T P(f, V, T(t)) dt \quad \text{s. t.} \quad \dot{T} = \alpha P(f, V, T) - \beta (T - T_{\text{amb}}),$$

wobei $T(t)$ die Chip-Temperatur ist und α, β thermische Transferkoeffizienten bezeichnen. Dieses Problem ist ein Optimal-Control-Problem und kann mit Pontrjaginschen Maximumprinzip oder numerischer Kollokation gelöst werden.

Hybride IoFET-Li-Ionen-Energiepolitik

Die KI-gestützte Akkuverwaltung [29] und das IoFET-DVFS-Modell [30] wurden bisher separat entwickelt. Eine integrierte Energiepolitik, die den Akkuzustand (State-of-Charge, State-of-Health) als Eingabe des DVFS-Reglers verwendet, könnte die Gesamtsystemlebensdauer signifikant verlängern. Formal lässt sich dies als *Markov-Decision-Process* (MDP) formulieren:

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right],$$

wobei der Zustandsraum $s_t = (f_t, V_t, \text{SoC}_t, T_t)$ und die Aktion a_t die Wahl der DVFS-Stufe ist. Der Reward R balanciert Performanz und Akkuverschleiß.

15.2.7 Dateisystem und Speicherverwaltung

Subgraph-basierte Deduplizierung auf NVMe-Speicher

Das Dateisystem-Deduplizierungsframework [25] nutzt Subgraph-Isomorphismus zur Erkennung strukturell ähnlicher Datenblöcke. Aktuelle Implementierungen operieren auf Block-Ebene; eine Erweiterung auf *semantische Deduplizierung* – d. h. inhaltsbezogene Ähnlichkeit auf Ebene von Dokumentstrukturen, Code-Abstrakt-Syntaxbäumen (AST) oder Vektordatenbanken (VADIS [41]) – würde erhebliche Speichereinsparungen ermöglichen. Für NVMe-SSDs mit 10^9 Datenobjekten ist eine parallele Subgraph-Verarbeitung auf PYMCA-8 mit $O(n^2 \log n)$ Komplexität erforderlich.

Persistente Speicherklassen (NVDIMM, CXL)

Compute Express Link (CXL) 3.0 ermöglicht Memory-Pooling zwischen mehreren Hosts. BS/EDF⁺ sollte CXL als neue Speicherklasse modellieren: CXL-Speicher hat höhere Latenz als DRAM, aber niedrigere als NVMe; das stochastische Archiv-Modell [23] muss um eine dritte Speicherebene (DRAM / CXL / NVMe) erweitert werden. Die optimale Batch-Timer-Strategie ändert sich dabei qualitativ: Für CXL-Zugriffe mit Poisson-Ankünften der Rate λ_{CXL} ergibt sich ein modifiziertes Optimierungsproblem für den Erwartungswert der Latenz.

15.2.8 Graphentheoretische Systemverifikation

HS-DFS-Erweiterung für dynamische Kernel-Graphen

Der Subgraph Algorithmus [21] und der HS-DFS-Algorithmus (Graph-of-Graphs, [20]) operieren aktuell auf statischen Graphen. Echtzeit-Kernel-Systeme sind jedoch dynamisch: Treiber werden geladen und entladen, IPC-Verbindungen werden aufgebaut und beendet. Eine Erweiterung auf *dynamische Graph-Ströme* (*graph streams*) mit Online-Subgraph-Erkennung in amortisierter $O(\log n)$ -Zeit pro Änderungsoperation ist ein wichtiger offener Forschungspunkt.

Boolesche Matrixmultiplikation für Cache-Kohärenz

Die Boolesche Matrixmultiplikation in $O(n^{2.37})$ [22] wurde bisher für TLB-Cache-Optimierung genutzt. Eine direkte Anwendung liegt in der *Cache-Kohärenz-Analyse* auf PYMCA-8: Die Abhängigkeitsmatrix $D \in \{0, 1\}^{n \times n}$ zwischen n Tasks wird als Boolesches Matrix-Produkt berechnet; Einträge $D_{ij} = 1$ signalisieren potenzielle Cache-Invalidierungen. Die schnelle BMM ermöglicht es, diese Analyse in jedem Scheduling-Zyklus ($\approx 1 \mu\text{s}$) neu auszuführen.

15.2.9 Sicherheitsarchitektur und Isolation

Capability-basierte Zugriffskontrolle

Das aktuelle BS/EDF⁺-Sicherheitsmodell basiert auf LWE-Adressierung (Satz 14.6) und Subgraph-Integritätsprüfung. Eine Erweiterung um *Capabilities* (im Sinne von seL4 oder CHERI) würde eine feingranulare Zugriffskontrolle auf Objektebene ermöglichen: Jede Capability $c = (o, p, \sigma)$ kodiert ein Objekt o , eine Permissionsmaske p und eine kryptographische Signatur σ (Dilithium-3). Der Subgraph Algorithmus prüft, ob der Capability-Graph des anfragenden Tasks ein gültiger Subgraph des Ressourcen-Graphen ist – eine natürliche Erweiterung von Satz 14.2 auf Sicherheitspolicies.

Isolierte Ausführungsumgebungen für AUTOSAR-Partitionen

ISO 26262 [7] fordert räumliche und zeitliche Isolation zwischen AUTOSAR-Partitionen (ASIL-A bis ASIL-D). BS/EDF⁺ kann diese Isolation durch eine Kombination aus (1) EDF⁺-Time-Partitioning (zeitliche Isolation), (2) LWE-gesicherter Speicheradressierung per Partition (räumliche Isolation) und (3) Subgraph-Kompatibilitätsprüfung vor jedem Treiberladen gewährleisten. Eine formale Verifikation dieser Drei-Ebenen-Isolation in Isabelle/HOL würde eine ASIL-D-Zertifizierung von BS/EDF⁺ ermöglichen – eine erhebliche industrielle Wirkung.

15.2.10 Systemzustandsklassifikation und Echtzeit-Diagnostik

Erweiterung der KS-Klassifikation auf fünf Ebenen

Das aktuelle Modell [24] unterscheidet vier Systemzustände KS-I (stabil) bis KS-IV (überlastet). Eine Erweiterung auf fünf Ebenen durch Hinzunahme eines *KS-V: Degradierter Betrieb* würde qualitativ neue Reaktionsstrategien ermöglichen: Im KS-V-Zustand reduziert der Kernel proaktiv die Anzahl aktiver Tasks auf eine sichere Teilmenge, die durch Lyapunov-Stabilität (Satz 14.4) garantiert stabil bleibt. Der Übergang KS-IV $\rightarrow$ KS-V wäre durch eine Hysterese-Bedingung geregelt, um Flutter-Effekte zu vermeiden.

Echtzeit-Spektralanalyse der Systemzustandsmatrix

Satz 14.4 zeigt, dass Lyapunov-Stabilität äquivalent zu $\max_i \operatorname{Re}(\lambda_i(A)) < 0$ ist. Eine kontinuierliche Online-Überwachung des Spektralradius $\rho(A)$ in Echtzeit erfordert effiziente inkrementelle Eigenwertberechnung. Mit der Borst-Golub-Reinsch-Methode für *rank-1-updates* der Systemmatrix $A \mapsto A + uv^T$ lassen sich Eigenwertänderungen in $O(n^2)$ statt $O(n^3)$ berechnen – ein wesentlicher Effizienzgewinn für den $n = 8$ Kern-Fall von PYMCA-8.

15.2.11 Langfristige Vision: BS/EDF⁺ als offenes Ökosystem

Open-Source-Kernel-Implementierung

Alle in dieser Arbeit beschriebenen Algorithmen und Konzepte sind implementierungsoffen und unter MIT-Lizenz auf gitlab.com/epp-group/ veröffentlicht. Die nächste Stufe ist ein ausführbarer Proof-of-Concept-Kernel in Rust:

1. **Bootloader:** 16-Bit-Realmode $\rightarrow$ 32-Bit-Protected-Mode $\rightarrow$ 64-Bit-Long-Mode, verifiziert in TLA⁺.

2. **EDF⁺-Scheduler:** Rust-no_std-Implementierung mit formaler Korrektheit durch Coq-Extraktion.
3. **Subgraph-Modul:** $O(n^3)$ -Treiber-Kompatibilitätsprüfung als Kernel-Subsystem.
4. **LWE-Speichertreiber:** CRYSTALS-Kyber-basierter Page-Table-Walk für Hochsicherheitsseiten.
5. **IoFET-DVFS-Regler:** Sigmoid-Kurvenparametrisierung mit Online-Temperaturkorrektur.

Interoperabilität mit Linux-Treibern via AUTOSAR-HAL

Eine vollständige Neuentwicklung aller Gerätetreiber ist mittelfristig nicht realistisch. Stattdessen kann BS/EDF⁺ über eine *Hardware Abstraction Layer* (HAL) kompatibel zu bestehenden Linux-Treibern (via AUTOSAR-HAL-Schicht, [27]) gemacht werden: Linux-Treiber werden als EDF⁺-Tasks mit konservativ geschätzten WCET-Werten modelliert; Satz 14.2 prüft Kompatibilität automatisch. Damit wird der Übergang von Linux zu BS/EDF⁺ zu einem inkrementellen Migrationspfad.

Referenzimplementierung für die Lehre

BS/EDF⁺ ist als Lehrgegenstand in Betriebssystem-Vorlesungen besonders geeignet, weil alle Konzepte – EDF-Scheduling, formale Verifikation, Kryptographie, Graphenalgorithmen – in einem kohärenten Rahmen zusammengeführt werden. Eine *QEMU-basierte Lehrumgebung* mit vorkompilierten Kernel-Images und schrittweisem Debugger würde Studierenden ermöglichen, jeden der sechs Hauptsätze dieser Arbeit direkt am laufenden System zu beobachten.

15.2.12 Gesellschaftliche und industrielle Perspektive

Sicherheitskritische Infrastruktur

Mit der zunehmenden Digitalisierung kritischer Infrastrukturen (Energienetze, Wasserversorgung, autonome Fahrzeuge) steigt der Bedarf an formal verifizierten Betriebssystemen dramatisch. BS/EDF⁺ bietet einen Weg, wie ein vollständig beweisbares Echtzeit-OS für ASIL-D-Systeme aussehen könnte – ein Beitrag, der weit über die akademische Forschung hinausgeht.

Software-Defined Vehicles

Die AUTOSAR-OTA-Architektur [26] und EDF⁺-basierte Treiber-Validierung (Satz 14.2) sind direkte Bausteine für Software-Defined Vehicles (SDV): Fahrzeuge, deren gesamte Funktionalität durch Software definiert und über-the-air aktualisiert wird. BS/EDF⁺ liefert das formale Fundament für die Sicherheitsnachweise, die regulatorische Zulassungsbehörden (KBA, UN-ECE WP.29) für SDV-Systeme fordern werden.

Quantensichere Infrastruktur bis 2035

Satz 14.6 positioniert BS/EDF⁺ als frühen Vertreter einer Generation von Betriebssystemen, die bereits heute gegen zukünftige Quantenangriffe gehärtet sind. Mit der bevorstehenden

Migration nationaler Sicherheitsbehörden auf Post-Quanten-Kryptographie (BSI TR-02102, NIST SP 800-208) ist die in dieser Arbeit entwickelte LWE-Speicheradressierung ein zeitgemäßer und zukunftssicherer Ansatz.

Bemerkung 15.1. Die in diesem Ausblick beschriebenen Forschungsziele sind nicht als sequenziell zu verstehen. Viele Pfade – formale Verifikation, ARM-Portierung, RL-basiertes DVFS, Post-Quanten-Migration – sind parallel verfolgbar und gegenseitig bereichernd. BS/EDF⁺ ist kein abgeschlossenes System, sondern ein lebendiges Forschungsprogramm, dessen innere Konsistenz durch die sechs Hauptsätze dieser Arbeit garantiert wird.

Literaturverzeichnis

- [1] C.L. Liu und J.W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM*, Bd. 20, Nr. 1, S. 46–61, 1973.
- [2] M.L. Dertouzos, “Control robotics: The procedural control of physical processes,” in *Proc. IFIP Congress*, S. 807–813, 1974.
- [3] W. Hoeffding, “Probability inequalities for sums of bounded random variables,” *Journal of the American Statistical Association*, Bd. 58, Nr. 301, S. 13–30, 1963.
- [4] T.H. Cormen, C.E. Leiserson, R.L. Rivest und C. Stein, *Introduction to Algorithms*, 4. Aufl. Cambridge, MA: MIT Press, 2022.
- [5] G.C. Buttazzo, *Hard Real-Time Computing Systems*, 3. Aufl. New York: Springer, 2011.
- [6] AUTOSAR Consortium, *AUTOSAR Classic Platform – Specification of Operating System*, Release R23-11, 2023.
- [7] ISO, *ISO 26262: Road Vehicles – Functional Safety*, 2. Aufl., 2018.
- [8] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer’s Manual*, Vol. 1–3, 2023.
- [9] IEC, *IEC 61158: Industrial Communication Networks – Fieldbus Specifications*, 2019.
- [10] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *Journal of the ACM*, Bd. 56, Nr. 6, S. 1–40, 2009.
- [11] P.W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proc. 35th Annual Symposium on Foundations of Computer Science*, S. 124–134, 1994.
- [12] J. Bos u. a., “CRYSTALS – Kyber: A CCA-secure module-lattice-based KEM,” in *Proc. IEEE EuroS&P*, S. 353–367, 2018.
- [13] L. Ducas u. a., “CRYSTALS – Dilithium: A lattice-based digital signature scheme,” *IACR Trans. Cryptographic Hardware and Embedded Systems*, Bd. 2018, Nr. 1, S. 238–268, 2018.
- [14] S.A. Cook, “The complexity of theorem-proving procedures,” in *Proc. 3rd Annual ACM STOC*, S. 151–158, 1971.
- [15] H.K. Khalil, *Nonlinear Systems*, 3. Aufl. Upper Saddle River, NJ: Prentice Hall, 2002.
- [16] T. Nipkow, L.C. Paulson und M. Wenzel, *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, Berlin: Springer, 2002.

- [17] L. Lamport, *Specifying Systems: The TLA⁺ Language and Tools*, Boston: Addison-Wesley, 2002.
- [18] S. Epp, *Optimaler Scheduling-Algorithmus EDF⁺: Prioritätsstrafe und stochastische Analyse bei 40 Tasks*, 2026. Verfügbar auf gitlab.com/epp-group/edfplus
- [19] S. Epp, *PYMCA-8: Python Multi-Core Architecture 8*, 2026. Verfügbar auf gitlab.com/epp-group/pymca8
- [20] S. Epp, *The Hierarchical Subgraph-DFS Algorithm*, 2026. Verfügbar auf gitlab.com/epp-group/subgraph
- [21] S. Epp, *Der Subgraph Algorithmus: Graph-Profile und $O(n^3)$ -Isomorphismus*, 2026. Verfügbar auf gitlab.com/epp-group/subgraph
- [22] S. Epp, *Boolesche Matrixmultiplikation in $O(n^{2.37})$* , 2026. Verfügbar auf gitlab.com/epp-group/bool-mm
- [23] S. Epp, *Das stochastische Archiv-Problem: Optimaler Batch-Timer*, 2026. Verfügbar auf gitlab.com/epp-group/archv
- [24] S. Epp, *Dynamische Systemzustandsklassifikation KS-I bis KS-IV*, 2026. Verfügbar auf gitlab.com/epp-group/sysstate
- [25] S. Epp, *Dateisystem-Deduplizierung via Subgraph-Isomorphismus*, 2026. Verfügbar auf gitlab.com/epp-group/dusgraph
- [26] S. Epp, *AUTOSAR OTA-Update-Architektur für Software-Defined Vehicles*, 2026. Verfügbar auf gitlab.com/epp-group/autsre
- [27] S. Epp, *AUTOSAR Python-Treiber-Framework*, 2026. Verfügbar auf gitlab.com/epp-group/autsrepy
- [28] S. Epp, *EHAЕ: EtherCAT Hardware Abstraction Extension*, 2026. Verfügbar auf gitlab.com/epp-group/ethercate
- [29] S. Epp, *KI-gestütztes Power Management Model für Lithium-Ionen-Akkumulatoren*, 2026. Verfügbar auf gitlab.com/epp-group/liionp
- [30] S. Epp, *Ionische Feldeffekttransistoren (IoFET) und Sigmoid-DVFS*, 2026. Verfügbar auf gitlab.com/epp-group/iono
- [31] S. Epp, *Post-Quanten-Kryptographie: LWE und Gitter-basierte Adressierung*, 2026. Verfügbar auf gitlab.com/epp-group/pqc
- [32] S. Epp, *Physikalische Grundlagen digitaler Schaltkreise*, 2026. Verfügbar auf gitlab.com/epp-group/physd
- [33] S. Epp, *Signaltheorie: Fourier-Transformation und Abtasttheorem*, 2026. Verfügbar auf gitlab.com/epp-group/signalth
- [34] S. Epp, *Mobile Betriebssystem-Dateisystem-Architektur*, 2026. Verfügbar auf gitlab.com/epp-group/mobde

- [35] L. Sha, R. Rajkumar und J. P. Lehoczky, “Priority inheritance protocols: An approach to real-time synchronization,” *IEEE Trans. Computers*, Bd. 39, Nr. 9, S. 1175–1185, 1990.
- [36] S. K. Baruah, L. E. Rosier und R. R. Howell, “Algorithms and complexity concerning the preemptive scheduling of periodic, real-time tasks on one processor,” *Real-Time Systems*, Bd. 2, Nr. 4, S. 301–324, 1990.
- [37] S. Vestal, “Preemptive scheduling of multi-criticality systems,” in *Proc. 28th IEEE RTSS*, S. 239–243, 2007.
- [38] A. Burns und R. Davis, *Mixed Criticality Systems – A Review*, v5.0, Univ. of York, 2013.
- [39] W. Feller, *An Introduction to Probability Theory and Its Applications*, Bd. 1, 3. Aufl. New York: Wiley, 1968.
- [40] G. Klein, K. Elphinstone, G. Heiser u. a., “seL4: Formal verification of an OS kernel,” in *Proc. 22nd ACM Symposium on Operating Systems Principles (SOSP)*, S. 207–220, 2009.
- [41] S. Epp, *Skalierbare Vektorisierung und KI-gestützte Strukturierung heterogener Großdaten: Ein Framework für On-Demand-Darstellung (VADIS)*, 2026. Verfügbar auf gitlab.com/epp-group/vadis